

High-Level Design (HLD)

Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Microservices Ready (Phase 2)

Proposed HLD Structure (20 Chapters)

Chapter 1 – Introduction, Vision, Objectives & Architecture Principles

- Purpose of HLD
 - Business Vision
 - Technical Vision
 - Scope
 - Stakeholders
 - Assumptions
 - Constraints
 - Design Principles
 - Enterprise Architecture Overview
 - Technology Stack Overview
-

Chapter 2 – Overall System Architecture

- Enterprise Architecture Diagram
 - High-Level Component Diagram
 - User Journey Overview
 - System Context Diagram
 - Internal Module Communication
 - Layered Architecture
 - Modular Monolith Design
 - Future Microservices Strategy
-

Chapter 3 – User Roles & Portal Architecture

- Independent Learner Portal

- Recruiter Portal
 - Super Admin Portal
 - Authentication Flow
 - Authorization Model
 - Permission Matrix
 - Role Hierarchy
 - Cross-Portal Interaction
-

Chapter 4 – Functional Module Architecture

- Authentication
- Gamification Engine
- Brain Gym
- Aptitude Arena
- Coding Arena
- SQL Arena
- Debug Hero
- AI Coach
- Resume Builder
- Placement Portal
- Reports
- Notifications

Module interaction diagrams and dependencies.

Chapter 5 – Infrastructure & Deployment Architecture

- Production Architecture
 - Server Topology
 - Docker Architecture
 - Nginx
 - SSL
 - Domain Structure
 - CDN Strategy (Future)
 - Cloud Deployment
 - Environment Architecture (Dev, QA, UAT, Prod)
-

Chapter 6 – Data Architecture

- Database Overview
 - PostgreSQL Architecture
 - Redis Architecture
 - MinIO Storage
 - Backup Strategy
 - Data Flow
 - Data Lifecycle
 - Archiving Strategy
-

Chapter 7 – Security Architecture

- JWT Authentication
 - RBAC
 - Permission Model
 - Password Security
 - Encryption
 - API Security
 - OWASP Controls
 - Audit Trail
 - Session Management
-

Chapter 8 – Integration Architecture

- AI Integration
 - Email Service
 - SMS Gateway
 - File Storage
 - Future Payment Gateway
 - Third-Party APIs
 - Webhooks
 - Event Integration
-

Chapter 9 – Scalability & Performance Architecture

- Horizontal Scaling
- Vertical Scaling
- Redis Strategy
- Load Balancing

- Database Optimization
 - Capacity Planning
 - High Availability
 - Disaster Recovery
-

Chapter 10 – DevOps & CI/CD Architecture

- Git Strategy
 - GitHub Actions
 - Docker Pipeline
 - Build Pipeline
 - Deployment Pipeline
 - Rollback
 - Monitoring
 - Logging
-

Chapter 11 – Monitoring & Observability

- Prometheus
 - Grafana
 - Logback
 - ELK (Future)
 - Health Checks
 - Metrics
 - Alerts
 - Incident Response
-

Chapter 12 – Business Process Architecture

- Student Journey
 - Recruiter Journey
 - Placement Flow
 - AI Workflow
 - Resume Workflow
 - Assessment Workflow
 - Gamification Workflow
-

Chapter 13 – Network Architecture

- Network Diagram
 - Firewall
 - Reverse Proxy
 - HTTPS
 - API Gateway (Future)
 - DNS
 - Internal Communication
-

Chapter 14 – Availability & Disaster Recovery

- Backup Strategy
 - Failover
 - Recovery Plan
 - RPO
 - RTO
 - Replication
 - DR Procedures
-

Chapter 15 – Enterprise Integration Roadmap

Future integrations:

- LMS
 - ERP
 - HRMS
 - Video Platform
 - Payment Gateway
 - WhatsApp
 - Microsoft Teams
 - Zoom
 - Google Meet
-

Chapter 16 – Non-Functional Architecture

- Performance
- Scalability

- Reliability
 - Availability
 - Security
 - Maintainability
 - Extensibility
 - Accessibility
-

Chapter 17 – Technology Decision Matrix

Why the platform uses:

- Spring Boot
- React
- Flutter
- PostgreSQL
- Redis
- MinIO
- Docker
- JWT
- Maven

Technology alternatives and trade-offs.

Chapter 18 – Risk Assessment & Mitigation

- Technical Risks
- Infrastructure Risks
- Security Risks
- AI Risks
- Deployment Risks
- Scalability Risks
- Operational Risks

Mitigation strategies for each.

Chapter 19 – Architecture Roadmap

Phase 1

- Modular Monolith
- Single Database
- Docker Deployment

Phase 2

- Microservices
- Kafka
- Kubernetes
- API Gateway

Phase 3

- Multi-region Deployment
- Auto Scaling
- Service Mesh
- Event-driven Architecture

Chapter 20 – HLD Summary & Enterprise Architecture Conclusion

- Architecture Summary
- Design Principles Recap
- System Readiness
- Implementation Strategy
- Long-term Evolution Plan
- Architecture Governance
- Final Recommendations

Deliverables of the HLD

The completed HLD will include:

- Enterprise Architecture Diagrams
- Component Diagrams
- Deployment Diagrams
- Network Architecture
- Infrastructure Architecture
- Data Flow Diagrams

- Integration Diagrams
- Security Architecture
- Scalability Architecture
- Disaster Recovery Design
- Technology Decision Records
- Future Evolution Roadmap

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 1 – Introduction, Vision, Objectives & Architecture Principles

Document Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Microservices Ready (Phase 2)

Technology Stack: Spring Boot, React, Flutter, PostgreSQL, Redis, MinIO, Docker

1.1 Introduction

The **Smart Education Platform** is a comprehensive, AI-powered placement preparation and recruitment ecosystem designed to connect **Independent Learners**, **Recruiters**, and **Platform Administrators** through a unified digital platform.

Unlike traditional Learning Management Systems (LMS), **Phase 1 is focused exclusively on placement readiness**. It provides learners with gamified assessments, AI-assisted guidance, resume building, placement tracking, and recruitment opportunities without including a learning/content delivery module.

The platform has been designed with a scalable, secure, modular, and cloud-ready architecture capable of supporting future expansion into a full digital education ecosystem.

1.2 Purpose of High-Level Design (HLD)

The High-Level Design (HLD) document provides the architectural blueprint for the Smart Education Platform. It bridges the gap between business requirements (PRD/SRS) and technical implementation (LLD).

The HLD defines:

- Overall system architecture
 - Major software components
 - Module interactions
 - Infrastructure architecture
 - Deployment strategy
 - Security architecture
 - Integration architecture
 - Scalability strategy
 - Technology decisions
 - Future architecture roadmap
-

1.3 Business Vision

The vision of the Smart Education Platform is to become an intelligent placement preparation ecosystem that empowers learners to continuously improve their skills while enabling recruiters to identify and hire job-ready candidates efficiently.

The platform aims to:

- Improve placement success rates
 - Increase learner engagement through gamification
 - Simplify recruitment workflows
 - Leverage AI for personalized career guidance
 - Provide measurable performance analytics
-

1.4 Technical Vision

The technical vision is to build a platform that is:

- Secure
- Highly scalable
- Modular
- Cloud-native ready
- API-first

- AI-enabled
- Mobile-first
- Easy to maintain
- Easy to extend

The architecture should allow smooth migration from a **Modular Monolith** to **Microservices** without significant redesign.

1.5 Business Objectives

The platform shall enable:

Independent Learners

- Practice daily
 - Improve placement readiness
 - Track progress
 - Earn rewards
 - Build resumes
 - Receive AI coaching
 - Apply for jobs
-

Recruiters

- Post jobs
 - Search candidates
 - Screen resumes
 - Schedule interviews
 - Manage offers
 - Monitor recruitment analytics
-

Super Administrators

- Manage platform users
- Configure gamification
- Maintain question banks
- Monitor platform health
- Generate reports
- Manage security and compliance

1.6 Phase 1 Scope

The following modules are included in Phase 1:

Authentication

- Registration
- Login
- JWT Authentication
- Profile Management

Independent Learner

- Dashboard
- Gamification Engine
- Brain Gym
- Aptitude Arena
- Coding Arena
- SQL Arena
- Debug Hero
- AI Coach
- Resume Builder
- Placement Portal
- Reports
- Notifications

Recruiter Portal

- Dashboard
 - Job Management
 - Candidate Search
 - Resume Screening
 - Interview Scheduling
 - Offer Management
 - Reports
-

Super Admin Portal

- User Management
 - Recruiter Management
 - Job Management
 - Question Bank
 - Gamification Management
 - AI Configuration
 - Reports
 - Monitoring
 - Audit Logs
 - Platform Configuration
-

1.7 Out of Scope (Phase 1)

The following features are intentionally excluded:

- Learning Management System (LMS)
- Video Courses
- Live Classes
- Mentor Portal
- Parent Portal
- Online Exams with Remote Proctoring
- Payment Gateway
- Marketplace
- Community Forums
- Multi-language Support
- Offline Learning Mode

These are planned for future phases.

1.8 Stakeholders

Stakeholder	Responsibility
Product Owner	Product vision and prioritization
Project Manager	Planning and delivery
Solution Architect	System architecture
Backend Team	Spring Boot development

Stakeholder	Responsibility
Frontend Team	React web development
Mobile Team	Flutter application
UI/UX Team	Design system and user experience
QA Team	Testing and quality assurance
DevOps Team	Deployment and infrastructure
Business Team	Recruiter and learner onboarding

1.9 User Roles

The platform supports three primary roles:

Independent Learner

Prepares for placements through assessments, AI guidance, resume creation, and job applications.

Recruiter

Manages job postings, candidate screening, interviews, and hiring.

Super Admin

Controls platform operations, configuration, monitoring, security, and reporting.

1.10 Architecture Goals

The architecture shall achieve:

- High Performance
- High Availability
- Scalability
- Security
- Maintainability
- Reliability
- Modularity

- Extensibility
 - Testability
 - Observability
-

1.11 Architecture Principles

The Smart Education Platform follows these principles:

Modular Design

Each business module is independently organized and loosely coupled.

Separation of Concerns

Presentation, business logic, and data access are separated into distinct layers.

API-First Development

All functionality is exposed through REST APIs, enabling seamless integration with web and mobile clients.

Security by Design

Security is integrated into every layer, including authentication, authorization, validation, encryption, and auditing.

Mobile-First Experience

Core user journeys are optimized for mobile devices while maintaining a full-featured web experience.

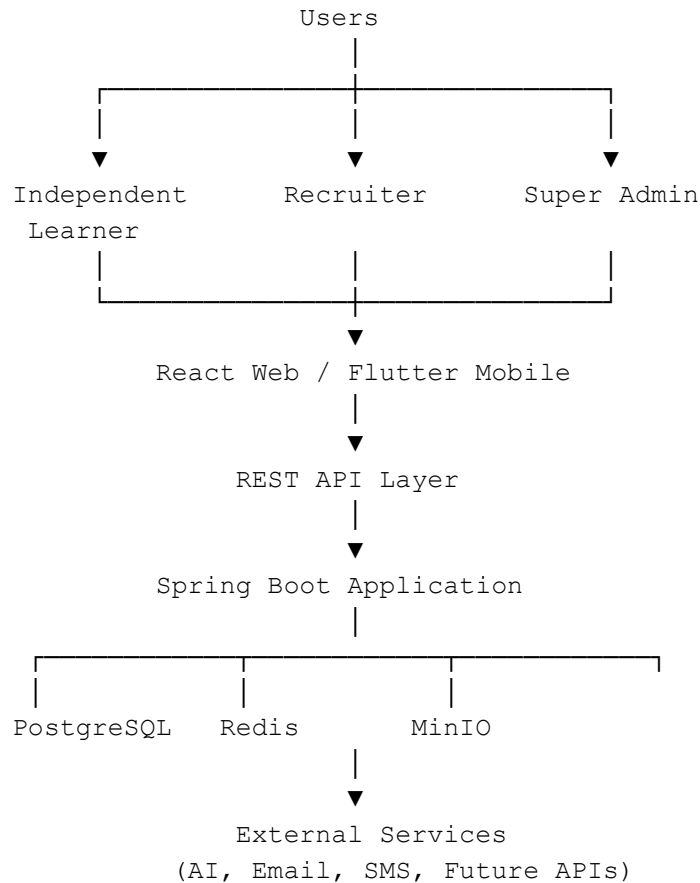
Cloud Readiness

The platform is designed for deployment in cloud environments with support for containerization and horizontal scaling.

Future-Proof Architecture

The solution is designed to evolve into a microservices architecture without major refactoring.

1.12 High-Level Architecture Overview



1.13 Technology Vision

Layer	Technology
Frontend Web	React
Mobile	Flutter
Backend	Spring Boot
Database	PostgreSQL
Cache	Redis
Object Storage	MinIO
Authentication	JWT

Layer	Technology
Build Tool	Maven
Containerization	Docker
Reverse Proxy	Nginx

1.14 Quality Attributes

The platform emphasizes:

- Performance
 - Scalability
 - Availability
 - Security
 - Reliability
 - Maintainability
 - Portability
 - Accessibility
 - Usability
 - Interoperability
-

1.15 Assumptions

- Internet connectivity is available.
 - Users access the platform through modern browsers or supported mobile devices.
 - External AI and notification services are operational.
 - PostgreSQL and Redis are available in production.
 - Docker is used for deployment.
-

1.16 Constraints

- Phase 1 uses a Modular Monolith architecture.
- Learning Module is excluded.
- Authentication is JWT-based.
- REST APIs are the only public interface.
- PostgreSQL is the primary database.
- Flutter is the mobile framework.

1.17 Success Criteria

The architecture will be considered successful when:

- All functional requirements are supported.
 - Response times meet defined performance targets.
 - Security controls are implemented.
 - The platform supports expected user loads.
 - Modules remain loosely coupled.
 - The system is ready for future microservices migration.
-

1.18 Expected Business Outcomes

- Higher learner engagement through gamification.
 - Faster placement preparation.
 - Improved recruiter productivity.
 - Increased placement success rate.
 - Reduced administrative effort.
 - Better visibility through analytics and reporting.
-

1.19 Document Relationship

The complete documentation suite consists of:

Business Vision



PRD (Product Requirements)



SRS (Software Requirements)

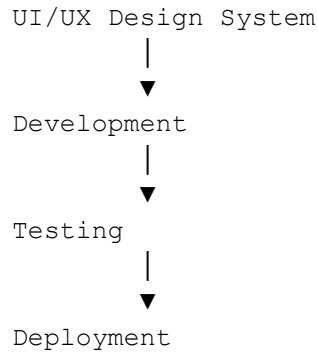


HLD (Enterprise Architecture)



LLD (Technical Design)





1.20 Chapter Summary

This chapter introduces the **High-Level Design (HLD)** for the Smart Education Platform and establishes the architectural vision, business objectives, scope, stakeholders, quality goals, and core design principles. It defines the platform as a secure, modular, AI-enabled, and cloud-ready placement preparation ecosystem with three primary portals—**Independent Learner, Recruiter, and Super Admin**—while explicitly excluding the Learning Module in Phase 1. The HLD serves as the architectural foundation that connects business requirements with technical implementation and guides the evolution of the platform toward a future microservices-based architecture.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 2 – Overall System Architecture

Document Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Microservices Ready (Phase 2)

2.1 Introduction

This chapter defines the overall enterprise architecture of the **Smart Education Platform**. It describes how all major components interact, how requests flow through the system, how data is processed, and how the platform is structured to support scalability, security, and maintainability.

The architecture is designed to support three primary user groups:

- Independent Learners
- Recruiters
- Super Administrators

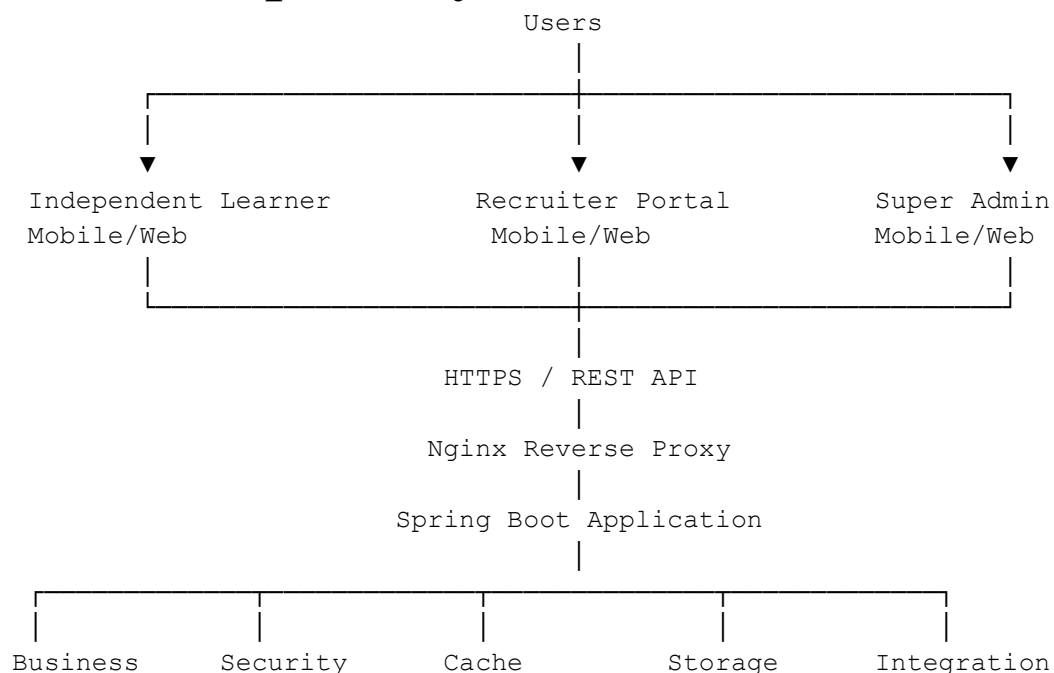
while maintaining a single, unified backend.

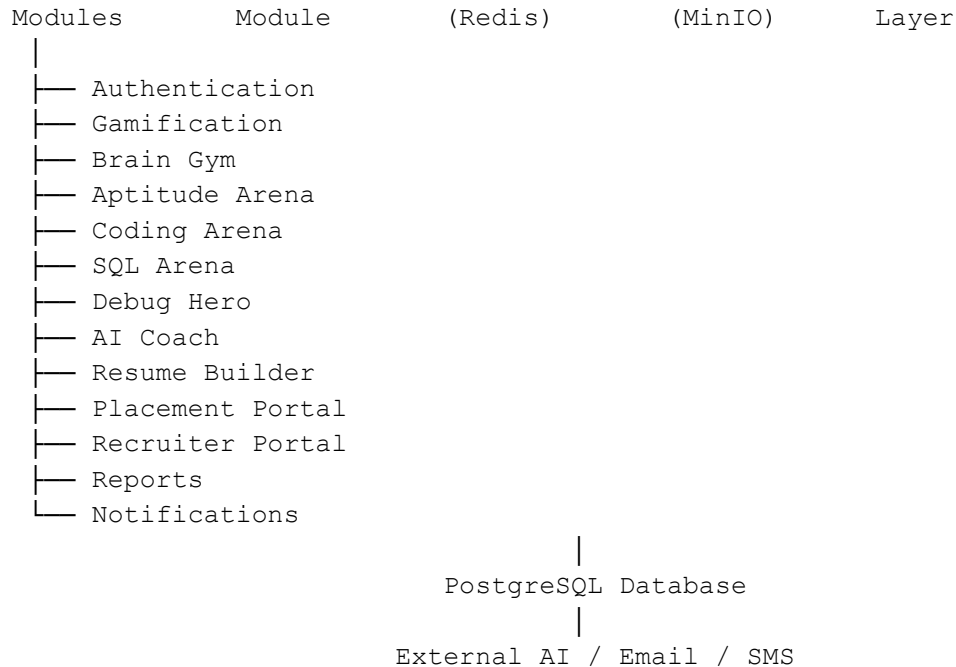
2.2 Architecture Objectives

The system architecture is designed to achieve:

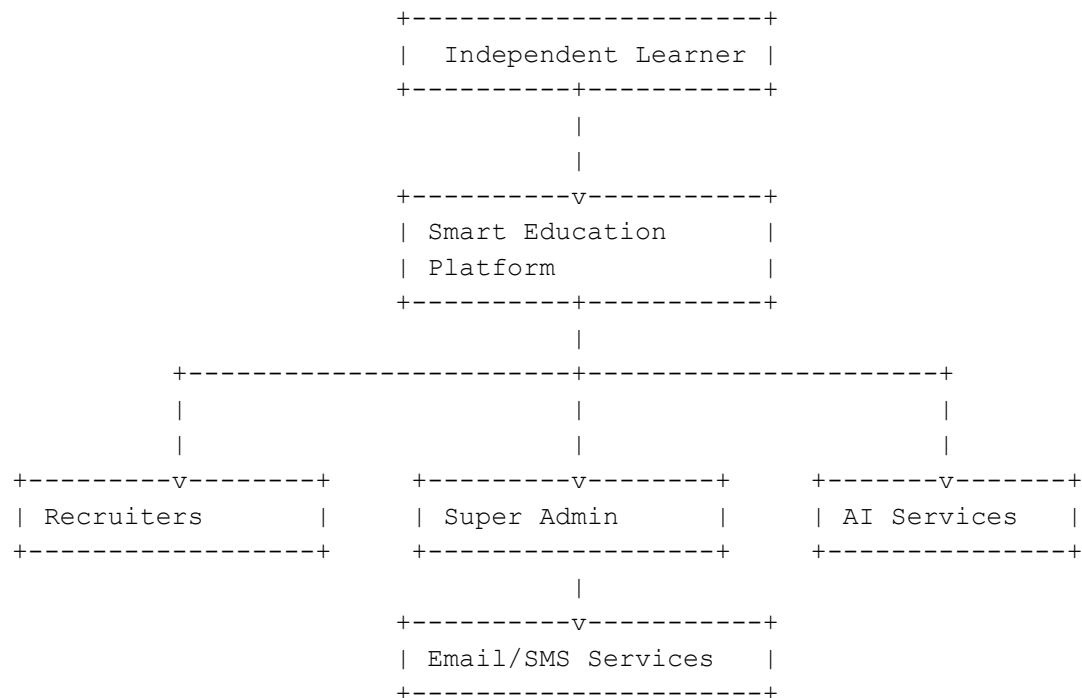
- Modular Design
 - High Availability
 - Horizontal Scalability
 - Cloud Readiness
 - API-First Development
 - Security by Design
 - Loose Coupling
 - High Performance
 - Easy Maintenance
 - Future Microservice Migration
-

2.3 Enterprise System Architecture



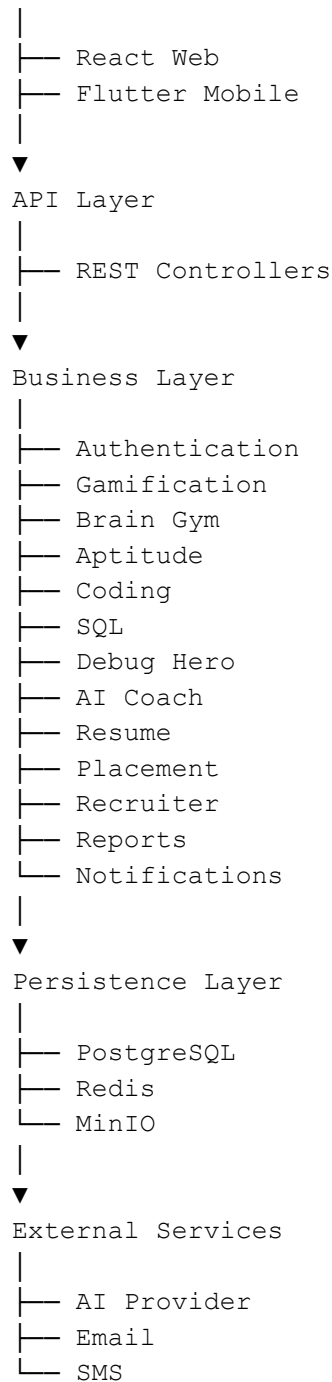


2.4 System Context Diagram



2.5 High-Level Component Architecture

Presentation Layer



2.6 Layered Architecture

The platform follows a layered architecture.

Presentation Layer

↓

API Layer

↓

Service Layer

↓

Repository Layer

↓

Database Layer

Each layer has a single responsibility and communicates only with adjacent layers.

2.7 Module Architecture

Business modules include:

Core Modules

- Authentication
 - User Management
 - Profile Management
-

Gamification Modules

- XP Engine
 - Coins
 - Badges
 - Leaderboards
 - Challenges
 - Daily Missions
-

Assessment Modules

- Brain Gym
- Aptitude Arena
- Coding Arena

- SQL Arena
 - Debug Hero
-

AI Modules

- AI Coach
 - Resume Review
 - Interview Guidance
-

Career Modules

- Resume Builder
 - Placement Portal
-

Administration Modules

- Recruiter Portal
 - Super Admin
 - Reports
 - Notifications
-

2.8 Request Processing Flow

User

↓

React / Flutter

↓

REST API

↓

JWT Authentication

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Response

↓

Frontend

2.9 Authentication Flow

Login

↓

JWT Generation

↓

Client Stores Token

↓

Every Request

↓

JWT Validation

↓

Role Validation

↓

Business Logic

↓

Response

2.10 Business Module Communication

Dashboard

|

|— Gamification

|— Reports

|— AI Coach

|— Placement

|— Notifications

|— Resume

|— Assessment Modules

Each module communicates through the service layer rather than direct database access.

2.11 Database Interaction

Controller

↓

Service

↓

Repository

↓

PostgreSQL

Redis is used for:

- OTP
 - Sessions
 - Leaderboards
 - Frequently accessed data
-

2.12 File Storage Flow

Upload Resume

↓

REST API

↓

Validation

↓

MinIO Storage

↓

Metadata in PostgreSQL

↓

Response

Stored files include:

- Resume PDFs
 - Profile Photos
 - Certificates
 - Company Logos
-

2.13 AI Integration Flow

User Question

↓

AI Coach

↓

Prompt Builder

↓

AI Provider

↓

Response Validation

↓

User

AI is used for:

- Resume Review
- Career Advice
- Interview Questions
- Coding Guidance

2.14 Notification Architecture

Business Event

↓

Notification Service

↓

Email

↓

SMS

↓

Push Notification (Future)

↓

In-App Notification

2.15 Reporting Architecture

Reports collect data from:

- Learners
- Recruiters
- Jobs
- Assessments
- Gamification
- AI Usage

Output formats:

- Dashboard Widgets
 - PDF
 - Excel
 - CSV
-

2.16 Modular Monolith Architecture

Phase 1 Architecture

Spring Boot Application

|— Authentication Module

|— Learner Module

|— Recruiter Module

|— Admin Module

|— Gamification Module

|— Assessment Module

|— AI Module

|— Placement Module

|— Notification Module

Each module is independently organized while sharing the same application and database.

2.17 Future Microservices Architecture

API Gateway

|

└─ Authentication Service

└─ User Service

└─ Assessment Service

└─ AI Service

└─ Gamification Service

└─ Placement Service

└─ Notification Service

└─ Reporting Service

Communication

- REST
- Event Streaming (Kafka - Future)

2.18 Technology Architecture

Layer	Technology
Web	React
Mobile	Flutter
Backend	Spring Boot
Security	Spring Security + JWT
Database	PostgreSQL
Cache	Redis
Object Storage	MinIO
Build	Maven
Reverse Proxy	Nginx

Layer	Technology
Container	Docker

2.19 Deployment Context

Internet

↓

Load Balancer

↓

Nginx

↓

Spring Boot

↓

PostgreSQL

↓

Redis

↓

MinIO

↓

Backup Server

2.20 External Integrations

Current Integrations

- AI Provider
- Email Service
- SMS Gateway

Future Integrations

- WhatsApp API
 - Payment Gateway
 - LinkedIn Jobs
 - Google Meet
 - Zoom
 - Microsoft Teams
 - Calendar APIs
-

2.21 Scalability Strategy

Phase 1

- Single Application
- Docker Deployment
- Redis Cache
- PostgreSQL

Phase 2

- Kubernetes
 - Auto Scaling
 - Kafka
 - API Gateway
 - Microservices
-

2.22 Security Boundaries

Security layers include:

- HTTPS
- JWT Authentication
- Role-Based Access Control (RBAC)
- Input Validation
- Rate Limiting
- Audit Logging
- Secure File Access
- API Authorization

2.23 Architecture Decision Records (ADR)

Decision	Reason
Modular Monolith	Faster development and simpler deployment
Spring Boot	Enterprise-grade backend framework
React	Rich, component-based web UI
Flutter	Single codebase for Android and iOS
PostgreSQL	Reliable relational database
Redis	High-performance caching
MinIO	Secure object storage
Docker	Consistent deployment environment

2.24 Non-Functional Support

The architecture supports:

- High Performance
 - Scalability
 - Availability
 - Reliability
 - Security
 - Maintainability
 - Extensibility
 - Testability
 - Observability
-

2.25 Architecture Principles Summary

The Smart Education Platform architecture is built on the following principles:

- Modular Design
- API-First Architecture
- Layered Architecture
- Secure by Default
- Cloud-Ready Deployment

- Mobile-First Experience
 - Reusable Components
 - Future Microservices Compatibility
 - High Cohesion and Low Coupling
-

2.26 Chapter Summary

This chapter defines the complete **Overall System Architecture** for the Smart Education Platform. It presents the enterprise-level structure of the application, including component organization, layered architecture, module interactions, request processing, authentication flow, storage, caching, reporting, AI integration, and deployment context. The platform adopts a **Modular Monolith** architecture in Phase 1, enabling rapid development while remaining fully prepared for a future migration to a scalable microservices ecosystem.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 3 – User Roles, Portal Architecture & Access Control

Document Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Microservices Ready (Phase 2)

3.1 Introduction

This chapter defines the **User Roles**, **Portal Architecture**, and **Access Control Model** for the Smart Education Platform.

The platform follows a **Role-Based Access Control (RBAC)** architecture to ensure that every authenticated user can only access resources and operations permitted by their assigned role.

The platform contains three independent portals:

- Independent Learner Portal
- Recruiter Portal

- Super Admin Portal

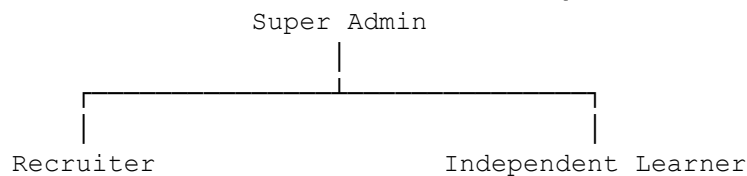
All portals share the same authentication service while maintaining strict authorization boundaries.

3.2 Objectives

The architecture shall:

- Support multiple user roles
 - Provide secure authentication
 - Enforce authorization rules
 - Prevent unauthorized access
 - Support future role expansion
 - Maintain complete auditability
-

3.3 User Role Hierarchy



Every role has its own permissions and dashboard.

3.4 User Types

Independent Learner

Purpose

- Placement preparation

Capabilities

- Practice assessments
- Participate in Brain Gym
- Coding Arena

- SQL Arena
- Debug Hero
- AI Coach
- Resume Builder
- Apply for Jobs
- View Reports
- Earn XP & Rewards

Cannot

- Post Jobs
 - Manage Users
 - Configure Platform
-

Recruiter

Purpose

- Hiring and recruitment

Capabilities

- Create Jobs
- Search Candidates
- Download Resumes
- Schedule Interviews
- Release Offers
- View Hiring Reports

Cannot

- Access Gamification Settings
 - Modify Questions
 - Manage Platform Configuration
-

Super Admin

Purpose

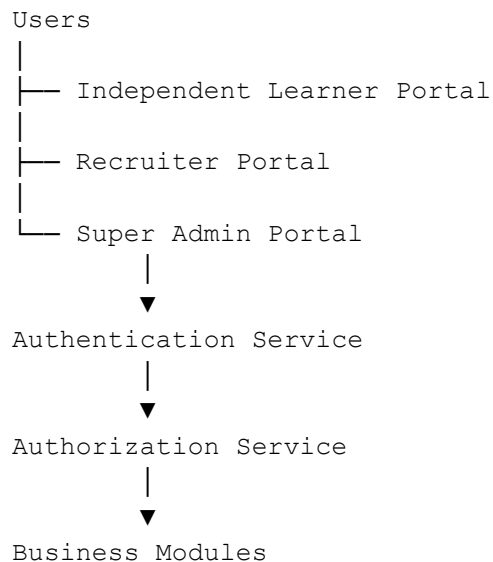
- Platform Administration

Capabilities

- Manage Users
- Manage Recruiters
- Manage Jobs
- Configure Gamification
- Manage Question Bank
- Configure AI
- Generate Reports
- Audit Logs
- Platform Settings
- Security Management

Has full administrative privileges.

3.5 Portal Architecture



3.6 Independent Learner Portal

Modules

Dashboard

↓

Gamification

↓

Brain Gym

↓

Aptitude Arena

↓

Coding Arena

↓

SQL Arena

↓

Debug Hero

↓

AI Coach

↓

Resume Builder

↓

Placement Portal

↓

Reports

↓

Profile

3.7 Recruiter Portal

Modules

Dashboard

↓

Job Management



Candidate Search



Resume Screening



Interview Management



Offer Management



Reports



Company Profile

3.8 Super Admin Portal

Modules

Dashboard



User Management



Recruiter Management



Job Management



Question Bank

↓

Gamification

↓

AI Configuration

↓

Reports

↓

Monitoring

↓

Audit Logs

↓

Platform Settings

3.9 Authentication Architecture

Authentication Process

Login

↓

Credentials Validation

↓

JWT Generation

↓

Refresh Token

↓

Client Storage

↓

Authenticated Requests

Authentication Methods

- Email & Password
 - Mobile & OTP (Future)
 - Google Sign-In (Future)
 - Enterprise SSO (Future)
-

3.10 Authorization Architecture

Authorization is enforced using **Role-Based Access Control (RBAC)**.

JWT

↓

Role Extraction

↓

Permission Validation

↓

Access Granted / Denied

Every request is validated before reaching the business logic.

3.11 Permission Matrix

Module	Learner	Recruiter	Super Admin
Dashboard	✓	✓	✓
Brain Gym	✓	×	View
Aptitude Arena	✓	×	View
Coding Arena	✓	×	View
SQL Arena	✓	×	View
Debug Hero	✓	×	View

Module	Learner	Recruiter	Super Admin
AI Coach	✓	✗	Configure
Resume Builder	✓	View	View
Placement Portal	✓	Manage	Manage
Job Management	✗	✓	✓
Candidate Search	✗	✓	✓
User Management	✗	✗	✓
Question Bank	✗	✗	✓
Gamification Settings	✗	✗	✓
Platform Settings	✗	✗	✓
Reports	Personal	Recruitment	Global

3.12 Resource Ownership

Learners may access:

- Their profile
- Their resume
- Their reports
- Their assessments

Recruiters may access:

- Their company jobs
- Their interviews
- Their candidates

Super Admins may access:

- All platform resources
-

3.13 Portal Navigation

Independent Learner

Dashboard

Arena

Jobs

AI Coach

Profile

Recruiter

Dashboard

Jobs

Candidates

Interviews

Profile

Super Admin

Dashboard

Users

Reports

Notifications

More

3.14 Cross-Portal Communication

Learner

↓

Placement Portal

↓

Recruiter

↓

Interview

↓

Offer

↓

Learner

Super Admin monitors every stage.

3.15 Access Control Flow

Request

↓

JWT Validation

↓

Role Validation

↓

Permission Validation

↓

Business Module

↓

Database

↓

Response

3.16 Session Management

Session Features

- JWT Authentication
 - Refresh Token
 - Token Expiry
 - Logout
 - Session Timeout
 - Token Revocation
-

3.17 Password Policy

Requirements

- Minimum 8 characters
- Uppercase letter
- Lowercase letter
- Number
- Special character

Future

- Password Expiry
 - Password History
 - MFA
-

3.18 Account Lifecycle

Registration

↓

Email Verification

↓

Active Account

↓

Profile Completion

↓

Placement Ready

↓

Inactive

↓

Suspended (if required)

↓

Deleted

3.19 Audit & Activity Tracking

Every important action is logged.

Examples

- Login
- Logout
- Password Change
- Resume Update
- Job Publish
- Recruiter Approval
- User Suspension
- Question Creation

Audit Data

- User ID
 - Role
 - IP Address
 - Timestamp
 - Module
 - Action
 - Status
-

3.20 Security Boundaries

Independent Learner

- Personal resources only

Recruiter

- Company-specific resources

Super Admin

- Global platform access

No portal can bypass role validation.

3.21 Future Multi-Tenant Architecture

Future enterprise deployments may support multiple organizations.

Platform

└─ Company A

└─ Company B

└─ Company C

└─ Educational Institution

Each tenant will have isolated users, jobs, reports, and configurations while sharing the same platform infrastructure.

3.22 Role Expansion Roadmap

Future roles

- Mentor
- Trainer
- College Administrator
- Placement Officer
- HR Manager
- Content Manager
- Parent
- External Evaluator

RBAC is designed to support additional roles without architectural changes.

3.23 Identity & Access Architecture

Users

↓

Authentication Service

↓

JWT Token

↓

Authorization Layer

↓

Role Validation

↓

Permission Check

↓

Business Module

3.24 Architecture Best Practices

- Enforce least-privilege access.
 - Centralize authentication and authorization.
 - Protect every API endpoint with role validation.
 - Log all security-sensitive operations.
 - Separate business responsibilities by portal.
 - Keep permission definitions configurable for future expansion.
-

3.25 Chapter Summary

This chapter defines the **User Roles, Portal Architecture, and Access Control** model of the Smart Education Platform. It establishes three secure portals—**Independent Learner, Recruiter, and Super Admin**—using a centralized authentication service and **Role-Based Access Control (RBAC)**. The architecture ensures that each role has clearly defined permissions, secure access boundaries, auditability, and a scalable foundation for future roles and multi-tenant deployments.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 5 – Infrastructure & Deployment Architecture

Document Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Cloud Native Microservices (Phase 2)

5.1 Introduction

This chapter defines the infrastructure and deployment architecture of the **Smart Education Platform**. It describes the production environment, deployment topology, network architecture, server infrastructure, containerization strategy, monitoring, backup, disaster recovery, and future cloud scalability.

The infrastructure is designed to be:

- Highly Available
 - Secure
 - Scalable
 - Containerized
 - Cloud Ready
 - Production Ready
-

5.2 Infrastructure Objectives

The infrastructure shall:

- Support 100,000+ learners in Phase 1
 - Achieve 99.9% uptime
 - Support horizontal scaling
 - Minimize downtime
 - Ensure secure deployments
 - Enable automated CI/CD
 - Support future Kubernetes migration
-

5.3 Environment Architecture

The platform uses four environments.

Developer Environment



Quality Assurance (QA)



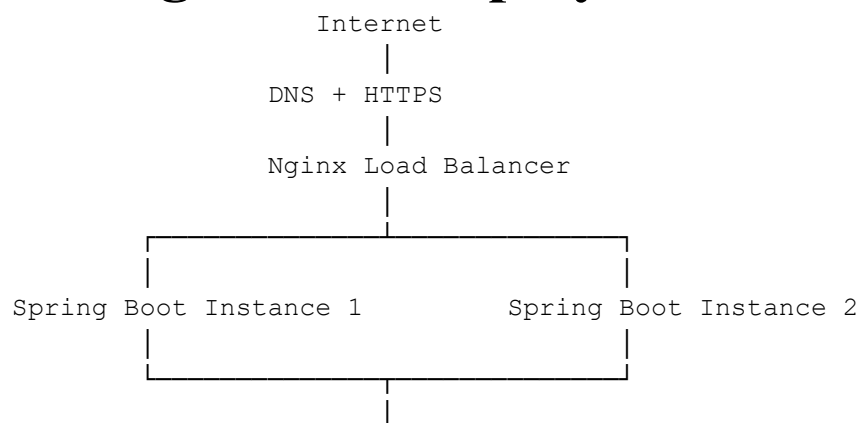
User Acceptance Testing (UAT)

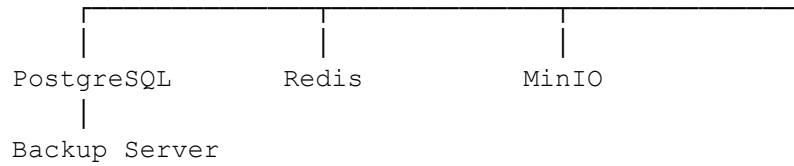


Production

Environment	Purpose
Development	Feature development
QA	Functional testing
UAT	Business validation
Production	Live platform

5.4 High-Level Deployment Architecture





5.5 Infrastructure Components

Core Infrastructure

- Nginx Reverse Proxy
- Spring Boot Application
- PostgreSQL
- Redis
- MinIO
- Docker
- SSL Certificate
- Monitoring Stack

5.6 Server Architecture

Application Server

Responsibilities

- REST APIs
- Business Logic
- Authentication
- AI Integration

Technology

- Spring Boot
- Java 21+
- Maven

Database Server

Responsibilities

- Store transactional data
- User profiles
- Reports
- Jobs
- Questions
- Resume metadata

Technology

- PostgreSQL
-

Cache Server

Technology

- Redis

Stores

- OTP
 - Sessions
 - Leaderboards
 - Dashboard Cache
 - Frequently Used Data
-

Object Storage

Technology

- MinIO

Stores

- Resume PDFs
 - Certificates
 - Profile Images
 - Company Logos
 - Documents
-

5.7 Network Architecture

```
Internet
|
Firewall
|
Load Balancer
|
Nginx
|
Spring Boot
|
PostgreSQL
```

5.8 Docker Architecture

Containers

Docker Host

```
|— Nginx
|
|— Spring Boot
|
|— PostgreSQL
|
|— Redis
|
|— MinIO
|
└— Monitoring
```

Each service runs independently.

5.9 Reverse Proxy

Technology

- Nginx

Responsibilities

- SSL Termination
- Load Balancing

- Request Routing
 - Compression
 - Static File Serving
 - Rate Limiting
-

5.10 SSL/TLS Architecture

Security

Browser

↓

HTTPS

↓

SSL Certificate

↓

Nginx

↓

Application

Supports

- TLS 1.2+
 - TLS 1.3
 - HSTS
 - Secure Cookies
-

5.11 Database Deployment

Production

Spring Boot

↓

PostgreSQL Primary

↓

Daily Backup

↓

Future Read Replica

Future

- Streaming Replication
- Failover

5.12 Redis Deployment

Used For

- Cache
- Session Storage
- OTP
- Leaderboards
- Dashboard Data

Future

- Redis Cluster
- Redis Sentinel

5.13 MinIO Deployment

Stores

- Resume
- Images
- Certificates
- Company Documents

Future

- Multi-node replication
- Object versioning

5.14 Monitoring Architecture

Monitoring Stack

Application

↓

Prometheus

↓

Grafana

↓

Alerts

↓

Administrator

Metrics

- CPU
- Memory
- JVM
- Database
- Redis
- API Response
- Disk Usage

5.15 Logging Architecture

Logging Framework

- Logback

Log Types

- Application Logs
- Security Logs

- Audit Logs
- API Logs
- Error Logs

Future

- ELK Stack
-

5.16 Backup Strategy

Daily

- PostgreSQL Backup

Weekly

- Full Backup

Monthly

- Archive Backup

Backup Includes

- Database
 - MinIO
 - Configuration
 - Docker Compose
 - Environment Files
-

5.17 Disaster Recovery

Recovery Objectives

Metric **Target**

RPO 15 Minutes

RTO 1 Hour

Recovery Includes

- Database Restore

- Object Storage Restore
 - Configuration Restore
 - Service Restart
-

5.18 Deployment Pipeline

Developer

↓

GitHub

↓

GitHub Actions

↓

Build

↓

Docker Image

↓

Deployment

↓

Production

5.19 CI/CD Architecture

Stages

- Build
- Unit Test
- Security Scan
- Package
- Docker Build
- Deploy
- Smoke Test

- Health Check
-

5.20 Infrastructure Security

Security Controls

- HTTPS
 - JWT Authentication
 - Firewall
 - Rate Limiting
 - Secure Headers
 - Environment Secrets
 - Database Encryption
 - Object Storage Policies
-

5.21 High Availability

Current

- Multiple Application Instances

Future

- Database Replication
- Redis Cluster
- Kubernetes
- Auto Scaling

Availability Target

99.9%

5.22 Scalability Strategy

Phase 1

- Vertical Scaling
- Docker

Phase 2

- Horizontal Scaling
 - Kubernetes
 - Auto Scaling
 - Load Balancer
-

5.23 Capacity Planning

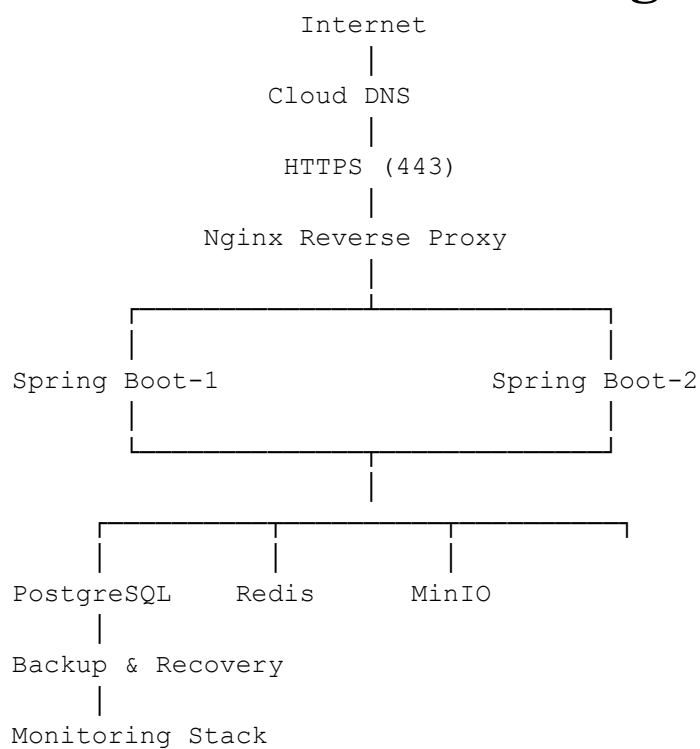
Expected Capacity

- 100,000+ Learners
- 5,000 Recruiters
- 50 Admins

Concurrent Users

- 5,000+
-

5.24 Infrastructure Diagram



5.25 Production Server Specification (Recommended)

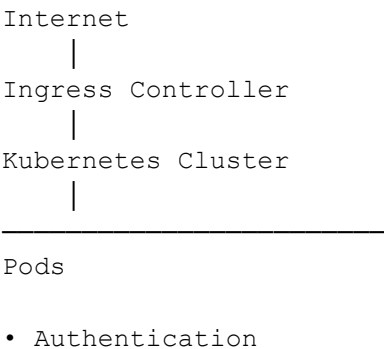
Component	Recommended Configuration
CPU	8 vCPU
RAM	32 GB
Storage	500 GB NVMe SSD
OS	Ubuntu Server 24.04 LTS
Java	JDK 21 LTS
Database	PostgreSQL 16
Redis	Redis 7
Object Storage	MinIO Latest
Reverse Proxy	Nginx Latest
Container Runtime	Docker Engine

5.26 Cloud Deployment Options

Supported Platforms

- AWS
- Microsoft Azure
- Google Cloud Platform (GCP)
- DigitalOcean
- Oracle Cloud Infrastructure (OCI)
- Cloudways (for small deployments)

5.27 Future Kubernetes Architecture



- Assessment
 - AI
 - Placement
 - Notification
 - Reports
-

Managed PostgreSQL
Redis Cluster
Object Storage

5.28 Infrastructure Best Practices

- Use Infrastructure as Code (IaC) for repeatable deployments.
 - Store secrets securely and never commit them to source control.
 - Enable automated daily backups and regularly verify restoration.
 - Monitor application health and infrastructure continuously.
 - Separate development, QA, UAT, and production environments.
 - Keep operating systems, containers, and dependencies updated.
 - Implement automated health checks and graceful rolling deployments.
-

5.29 Chapter Summary

This chapter defines the **Infrastructure & Deployment Architecture** for the Smart Education Platform. It provides a secure, scalable, and production-ready deployment model using **Spring Boot, PostgreSQL, Redis, MinIO, Docker, and Nginx**, supported by monitoring, automated backups, disaster recovery, and CI/CD. The architecture supports Phase 1 as a modular monolith while preparing the platform for future migration to Kubernetes-based microservices with high availability and auto-scaling capabilities.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 6 – Data Architecture

Document Version: 1.0

Architecture Style: Centralized Relational Database + Distributed Cache + Object Storage

6.1 Introduction

The **Data Architecture** defines how information is collected, stored, secured, processed, archived, and retrieved across the Smart Education Platform.

The architecture supports:

- High-performance transaction processing
- Secure data storage
- AI-driven analytics
- Enterprise reporting
- Future big-data expansion

The platform uses three storage technologies:

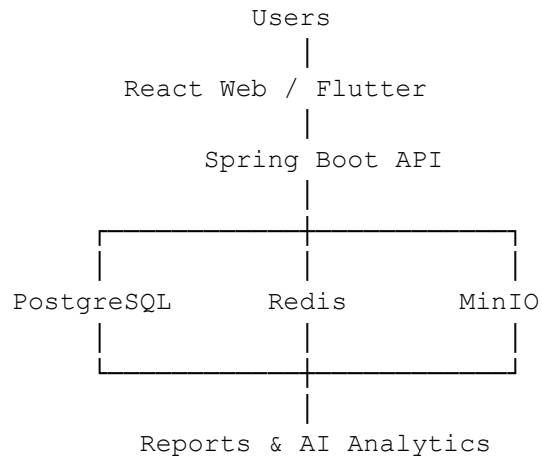
- **PostgreSQL** – Primary Relational Database
 - **Redis** – High-Speed Cache & Session Store
 - **MinIO** – Object Storage
-

6.2 Data Architecture Objectives

The architecture shall provide:

- Data Integrity
 - High Availability
 - ACID Transactions
 - Secure Storage
 - Fast Retrieval
 - Scalability
 - Backup & Recovery
 - Auditability
 - Future Analytics Readiness
-

6.3 Enterprise Data Architecture



6.4 Data Classification

Master Data

Examples

- Users
 - Roles
 - Permissions
 - Colleges
 - Companies
 - Skills
 - Technologies
 - Categories
-

Transaction Data

Examples

- Job Applications
 - Coding Submissions
 - SQL Attempts
 - Brain Gym Results
 - Aptitude Results
 - Resume Updates
 - AI Conversations
-

Configuration Data

Examples

- XP Rules
 - Badge Rules
 - Notification Templates
 - Email Settings
 - AI Prompt Templates
-

Audit Data

Examples

- Login History
 - API Logs
 - User Activity
 - Admin Actions
 - Recruiter Actions
-

6.5 Database Architecture

PostgreSQL

|— Authentication Schema

|— User Schema

|— Assessment Schema

|— Gamification Schema

|— Resume Schema

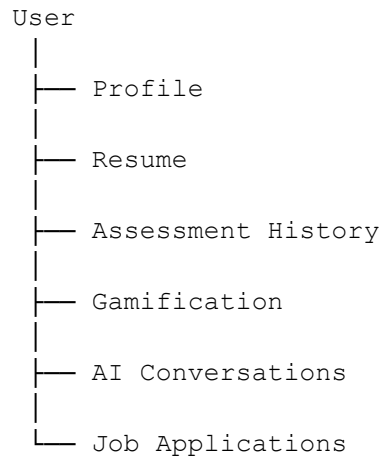
|— Placement Schema

|— Recruiter Schema

|— Reporting Schema

|— Audit Schema

6.6 High-Level Data Model



6.7 Core Business Entities

Authentication

- User
 - Role
 - Permission
 - RefreshToken
-

Learner

- LearnerProfile
 - Education
 - Skill
 - Project
 - Certificate
-

Assessment

- BrainGymAttempt
- AptitudeAttempt
- CodingSubmission
- SQLSubmission

- DebugSubmission
-

Gamification

- XPHistory
 - CoinHistory
 - Badge
 - Level
 - Leaderboard
 - Challenge
-

Resume

- Resume
 - Experience
 - Project
 - Skill
 - Achievement
-

Placement

- Job
 - Application
 - Interview
 - Offer
-

Recruiter

- Company
 - Recruiter
 - HiringPipeline
-

Reports

- Dashboard
- Analytics

- Statistics
-

Audit

- AuditLog
 - LoginHistory
 - ActivityLog
-

6.8 Data Flow Architecture

User

↓

REST API

↓

Business Service

↓

Repository

↓

PostgreSQL

↓

Redis Cache

↓

Response

6.9 Redis Data Architecture

Redis stores:

- Login Sessions
- JWT Blacklist

- OTP
- Dashboard Cache
- Leaderboards
- Frequently Used Questions
- AI Cache
- Configuration Cache

Example TTL

Data	TTL
OTP	5 Minutes
Dashboard	15 Minutes
Leaderboard	10 Minutes
AI Response	30 Minutes
Session	24 Hours

6.10 Object Storage Architecture

MinIO stores:

- Resume PDF
- Certificates
- Company Logo
- Profile Photo
- Export Reports
- Question Images

Metadata is stored in PostgreSQL while binary files remain in MinIO.

6.11 Data Lifecycle

Create

↓

Validate

↓

Store

↓

Update

↓

Archive

↓

Delete

Soft delete is preferred for business records.

6.12 Master Data Management

Master Data includes:

- Skills
- Colleges
- Companies
- Technologies
- Categories
- Difficulty Levels

Characteristics

- Centrally managed
 - Version controlled
 - Rarely modified
-

6.13 Transaction Data Management

Transactional entities include:

- Coding Attempts
- SQL Attempts
- Brain Gym Results
- AI Requests
- Resume Updates
- Job Applications

These require:

- ACID compliance
 - Full auditability
 - Accurate timestamps
-

6.14 Reporting Data

Reports are generated from:

- Assessments
- Jobs
- Recruiters
- AI Usage
- Gamification
- Placement Statistics

Export Formats

- PDF
 - Excel
 - CSV
-

6.15 Data Relationships

User

↓

Resume

↓

Job Application

↓

Interview

↓

Offer

Assessment relationships

User

↓

Coding

↓

XP

↓

Leaderboard

6.16 Data Security

Security controls

- AES Encryption (Sensitive Data)
 - HTTPS
 - JWT Authentication
 - Row-Level Authorization
 - Password Hashing
 - Secure File URLs
-

6.17 Data Privacy

Sensitive Information

- Passwords
- Refresh Tokens
- Email
- Mobile
- Resume Documents

Protection

- Encryption
- Access Control
- Audit Logging

- Secure Backups
-

6.18 Backup Strategy

Daily

- Incremental Backup

Weekly

- Full Backup

Monthly

- Archive Backup

Backup includes

- PostgreSQL
 - Redis Snapshot
 - MinIO
 - Configuration Files
-

6.19 Data Retention Policy

Data Type	Retention
Audit Logs	7 Years
Login History	1 Year
Job Applications	5 Years
AI Conversations	2 Years
Notifications	6 Months
Reports	5 Years

6.20 Archival Strategy

Archive

- Old Reports

- Expired Jobs
- Closed Interviews
- Old Notifications
- Audit Logs

Storage

- Archive Database
 - Cold Storage
-

6.21 Data Synchronization

Synchronization between:

- PostgreSQL ↔ Redis
- PostgreSQL ↔ MinIO
- Database ↔ Reports
- Database ↔ AI

Updates occur through application services to maintain consistency.

6.22 Data Quality

Validation Rules

- Mandatory fields
 - Unique constraints
 - Foreign key validation
 - Business rule validation
 - Duplicate detection
-

6.23 Performance Optimization

Techniques

- Indexing
- Query Optimization
- Pagination

- Redis Caching
 - Batch Processing
 - Read-only Transactions
 - Lazy Loading
 - Connection Pooling
-

6.24 Future Data Architecture

Future enhancements include:

- Read Replicas
 - Data Warehouse
 - Data Lake
 - Kafka Event Streaming
 - Real-Time Analytics
 - Machine Learning Pipelines
 - Business Intelligence Platform
-

6.25 Architecture Best Practices

- Normalize transactional data.
 - Cache frequently accessed information.
 - Store files outside the relational database.
 - Encrypt sensitive information.
 - Use soft deletes where business history is required.
 - Implement automated backup verification.
 - Maintain audit trails for critical operations.
 - Design schemas for future scalability.
-

6.26 Chapter Summary

This chapter defines the **Data Architecture** for the Smart Education Platform. It establishes a secure, scalable, and enterprise-ready data foundation using **PostgreSQL** for transactional data, **Redis** for caching and session management, and **MinIO** for object storage. The architecture covers data classification, lifecycle management, master and transactional data, security, backup, archival, retention, and future analytics readiness, ensuring reliable data management across all platform modules.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 7 – Security Architecture

Document Version: 1.0

Security Model: Zero Trust + Defense in Depth

7.1 Introduction

Security is a fundamental architectural pillar of the Smart Education Platform. Every request, API, user, module, and data asset is protected through multiple security layers.

The platform follows **Zero Trust Architecture**, where no user, service, or request is trusted by default. Every request must be authenticated, authorized, validated, and audited before accessing any protected resource.

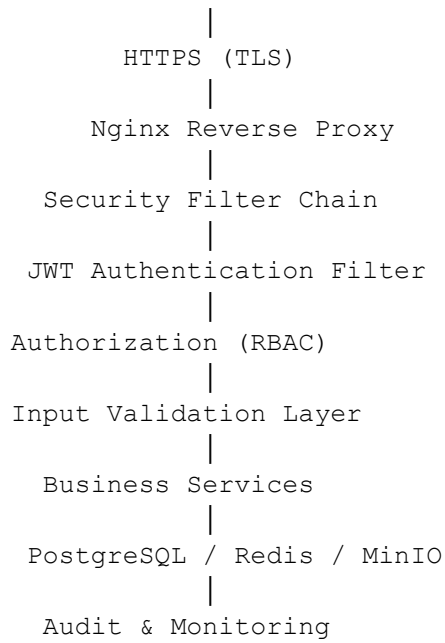
7.2 Security Objectives

The security architecture shall provide:

- Confidentiality
 - Integrity
 - Availability
 - Authentication
 - Authorization
 - Accountability
 - Non-Repudiation
 - Privacy Protection
 - Secure Communication
 - Continuous Monitoring
-

7.3 Security Architecture Overview

Users



7.4 Defense-in-Depth Model

The platform secures every layer.

Network Security

↓

Transport Security

↓

Authentication

↓

Authorization

↓

Application Security

↓

Database Security

↓

File Security

↓

Monitoring & Audit

7.5 Authentication Architecture

Supported Authentication

Phase 1

- Email + Password
- JWT Authentication
- Refresh Token

Future

- Mobile OTP
 - Google Login
 - LinkedIn Login
 - Microsoft Login
 - Multi-Factor Authentication (MFA)
 - Enterprise SSO
-

7.6 Authentication Flow

User Login

↓

Credential Validation

↓

Password Verification

↓

JWT Generation

↓

Refresh Token

↓

Secure Response

↓

Authenticated APIs

7.7 JWT Architecture

JWT Contains

- User ID
- Email
- Role
- Permissions
- Token ID
- Issued Time
- Expiry Time

Never Store

- Password
- Sensitive Personal Data

Recommended Expiration

Token	Expiry
-------	--------

Access Token	30 Minutes
--------------	------------

Refresh Token	7 Days
---------------	--------

7.8 Refresh Token Strategy

Access Token Expired

↓

Refresh Token Validation

↓

Generate New Access Token

↓

Continue Session

Benefits

- Better security
 - Short-lived access tokens
 - Reduced login frequency
-

7.9 Authorization Architecture

Role-Based Access Control (RBAC)

JWT

↓

Extract Role

↓

Load Permissions

↓

Validate Access

↓

Execute Request

Roles

- Independent Learner
 - Recruiter
 - Super Admin
-

7.10 Permission Model

Examples

Independent Learner

- Practice Challenges
- Apply Jobs
- Build Resume

Recruiter

- Post Jobs
- Search Candidates
- Schedule Interviews

Super Admin

- Full Platform Access
 - Manage Users
 - Configure Platform
-

7.11 Password Security

Password Rules

- Minimum 8 Characters
- Uppercase Letter
- Lowercase Letter
- Number
- Special Character

Storage

- BCrypt Hashing
- Salted Passwords

Passwords are never stored in plain text.

7.12 API Security

All APIs shall:

- Require HTTPS

- Validate JWT
 - Validate User Role
 - Validate Input
 - Apply Rate Limiting
 - Log Security Events
-

7.13 HTTPS & TLS

Communication

Browser

↓

HTTPS

↓

TLS 1.3

↓

Server

Supports

- TLS 1.2+
 - TLS 1.3
 - HSTS
 - Perfect Forward Secrecy
-

7.14 Data Encryption

Data in Transit

- HTTPS
 - TLS
-

Data at Rest

Sensitive data encrypted using AES-256.

Examples

- API Keys
 - Secrets
 - Refresh Tokens
 - Sensitive Configuration
-

7.15 Database Security

Security Controls

- Least Privilege Access
 - Database Roles
 - Prepared Statements
 - Parameterized Queries
 - Connection Encryption
 - Backup Encryption
-

7.16 Redis Security

Security

- Password Protected
 - Private Network Access
 - TLS (where supported)
 - No Public Exposure
-

7.17 Object Storage Security

MinIO Security

- Private Buckets
- Signed URLs
- Expiring Download Links
- Access Policies
- Server-side Encryption

7.18 Secure File Upload

Validation

- File Size
- File Type
- Virus Scan (Future)
- Filename Sanitization

Allowed Files

- PDF
 - JPG
 - PNG
-

7.19 Input Validation

Validate

- Request Body
- Query Parameters
- Path Variables
- File Uploads

Protection Against

- SQL Injection
 - XSS
 - Command Injection
 - Invalid Data
-

7.20 OWASP Top 10 Mitigation

Threat	Protection
Broken Access Control	RBAC
Cryptographic Failures	AES-256, TLS
Injection	Parameterized Queries

Threat	Protection
Insecure Design	Secure Coding Standards
Security Misconfiguration	Hardened Configuration
Vulnerable Components	Dependency Scanning
Authentication Failures	JWT + BCrypt
Software Integrity Failures	Signed Builds
Logging Failures	Audit Logs
SSRF	URL Validation

7.21 HTTP Security Headers

Headers

- Strict-Transport-Security
 - Content-Security-Policy
 - X-Content-Type-Options
 - X-Frame-Options
 - Referrer-Policy
 - Permissions-Policy
-

7.22 Rate Limiting

Limits

Login API

- 5 Requests / Minute

General API

- 100 Requests / Minute

AI API

- Configurable

Benefits

- Prevent brute-force attacks
 - Reduce abuse
 - Protect infrastructure
-

7.23 Session Management

Features

- JWT
 - Refresh Token
 - Logout
 - Token Revocation
 - Automatic Expiry
 - Session Timeout
-

7.24 Audit Logging

Log

- Login
- Logout
- Password Change
- Job Publish
- Resume Update
- Admin Actions
- Security Violations

Captured Data

- User
 - Role
 - IP Address
 - Browser
 - Timestamp
-

7.25 Security Monitoring

Monitor

- Failed Logins
 - API Errors
 - Suspicious Requests
 - Token Abuse
 - Database Errors
 - File Upload Failures
-

7.26 Incident Response

Threat Detected

↓

Alert Generated

↓

Investigation

↓

Containment

↓

Recovery

↓

Post-Incident Review

7.27 Secret Management

Secrets

- JWT Secret
- Database Password
- Redis Password
- SMTP Password
- AI API Keys

Best Practices

- Environment Variables
 - Secret Manager (Future)
 - Never commit secrets to Git
-

7.28 Security Compliance

The platform aligns with:

- OWASP Top 10
- Secure Coding Guidelines
- Password Security Best Practices
- Data Privacy Principles
- Audit Logging Requirements

Future

- ISO 27001 Readiness
 - SOC 2 Readiness
-

7.29 Future Security Enhancements

Phase 2

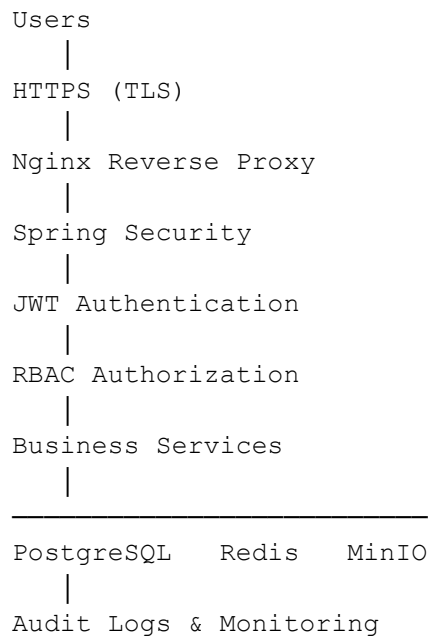
- Multi-Factor Authentication (MFA)
 - Biometric Login
 - Single Sign-On (SSO)
 - OAuth2/OpenID Connect
 - API Gateway
 - Web Application Firewall (WAF)
 - SIEM Integration
 - Device Trust
 - Risk-Based Authentication
-

7.30 Security Best Practices

- Apply the principle of least privilege.

- Protect all APIs with authentication and authorization.
 - Encrypt sensitive data in transit and at rest.
 - Validate every user input.
 - Use secure password hashing algorithms.
 - Enable comprehensive audit logging.
 - Rotate secrets and credentials regularly.
 - Perform regular vulnerability assessments.
 - Keep all software dependencies updated.
 - Monitor security events continuously.
-

7.31 Security Architecture Diagram



7.32 Chapter Summary

The **Security Architecture** provides a comprehensive, enterprise-grade security framework for the Smart Education Platform. It combines **Zero Trust principles**, **JWT-based authentication**, **Role-Based Access Control (RBAC)**, **secure communication**, **data encryption**, **OWASP Top 10 protections**, **audit logging**, and **continuous monitoring** to safeguard users, applications, and data. The architecture is designed to meet current Phase 1 requirements while providing a scalable foundation for future enhancements such as MFA, SSO, WAF, and advanced security monitoring.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 8 – Integration Architecture

Document Version: 1.0

Architecture Style: API-First, Service-Oriented Integration

8.1 Introduction

The **Integration Architecture** defines how the Smart Education Platform communicates with internal modules and external systems. It provides a standardized, secure, and scalable approach for integrating AI services, email providers, SMS gateways, object storage, and future enterprise services.

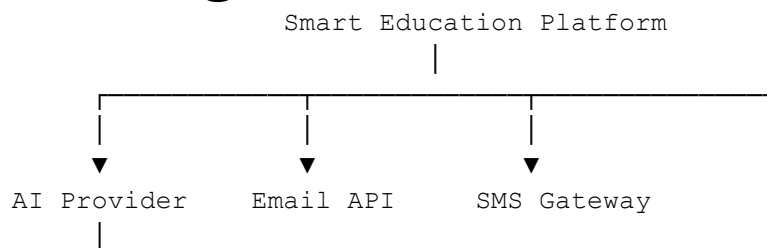
The architecture follows an **API-First** strategy, enabling seamless communication between the platform and third-party systems while maintaining loose coupling and fault tolerance.

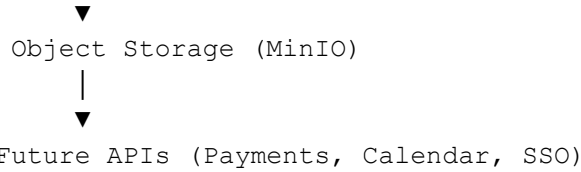
8.2 Integration Objectives

The integration architecture shall:

- Enable secure communication with external services.
 - Standardize API interactions.
 - Support asynchronous processing where required.
 - Handle failures gracefully.
 - Minimize dependencies between modules.
 - Simplify future integrations.
-

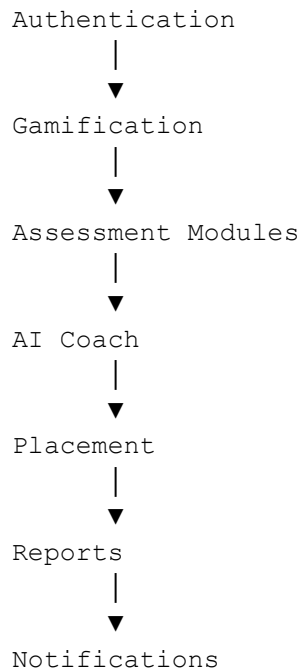
8.3 Integration Architecture Overview





8.4 Internal Integration Architecture

Business modules communicate only through the Service Layer.



Benefits

- Loose coupling
 - Better maintainability
 - Easier testing
 - Future microservice compatibility
-

8.5 AI Integration Architecture

AI is used for:

- Career Guidance
- Resume Review

- Resume Improvement Suggestions
- Interview Preparation
- Coding Assistance
- SQL Assistance
- Debugging Assistance

Flow

User

↓

AI Coach

↓

Prompt Builder

↓

AI Provider API

↓

Response Formatter

↓

User

8.6 AI Request Pipeline

Every AI request passes through:

1. User Validation
 2. Prompt Construction
 3. Context Injection
 4. AI Request
 5. Response Validation
 6. Response Formatting
 7. Audit Logging
 8. Cache Storage (Optional)
-

8.7 Email Integration

Email is used for:

- Registration Verification
- Password Reset
- Job Application Confirmation
- Interview Invitations
- Offer Letters
- System Notifications

Architecture

Business Event

↓

Notification Service

↓

Email Provider

↓

User

8.8 SMS Integration

SMS notifications include:

- OTP
- Interview Reminder
- Password Reset
- Important Alerts

Flow

Notification Service

↓

SMS Gateway

↓

Mobile Network

↓

User

8.9 Object Storage Integration

Technology

- MinIO

Stores

- Resume PDFs
- Profile Photos
- Company Logos
- Certificates
- Exported Reports

Flow

Upload

↓

Validation

↓

MinIO

↓

Metadata → PostgreSQL

8.10 REST API Integration

External integrations use REST APIs.

Supported Methods

- GET
- POST
- PUT
- PATCH
- DELETE

Standards

- HTTPS
 - JSON
 - JWT Authentication
 - Versioned APIs
-

8.11 Integration Security

Security controls

- HTTPS
 - TLS 1.3
 - JWT Authentication
 - API Keys
 - Request Validation
 - Response Validation
 - Input Sanitization
 - Rate Limiting
-

8.12 Integration Error Handling

Possible failures

- Timeout
- Invalid Response
- Authentication Failure
- Service Unavailable
- Network Failure

Handling Strategy

- Retry

- Circuit Breaker
 - Logging
 - User-friendly Error Messages
 - Alert Generation
-

8.13 Retry Strategy

Retry for:

- Temporary Network Failures
- HTTP 5xx Errors
- Timeout Errors

Policy

- Maximum 3 retries
 - Exponential Backoff
 - Jitter to reduce retry storms
-

8.14 Circuit Breaker Pattern

Request

↓

External Service

↓

Failure Threshold

↓

Circuit Open

↓

Fallback Response

↓

Automatic Recovery

Benefits

- Prevent cascading failures
 - Improve platform stability
 - Faster recovery
-

8.15 Notification Integration

Notification channels

- In-App
- Email
- SMS

Future

- Push Notifications
 - WhatsApp
 - Microsoft Teams
 - Slack
-

8.16 Webhook Architecture

Supported Events

- Job Published
- Resume Uploaded
- Interview Scheduled
- Offer Released

Flow

Business Event

↓

Webhook Dispatcher

↓

External System

Future webhook security

- HMAC Signature
 - Timestamp Validation
 - Replay Protection
-

8.17 API Gateway (Future)

Phase 2

Clients

↓

API Gateway

↓

Authentication

↓

Microservices

Gateway Responsibilities

- Routing
 - Authentication
 - Rate Limiting
 - Logging
 - Monitoring
 - API Versioning
-

8.18 Event-Driven Integration (Future)

Technology

- Apache Kafka

Example Events

- User Registered

- Job Applied
- Resume Updated
- Badge Earned
- Interview Scheduled

Benefits

- Loose coupling
 - Scalability
 - Asynchronous communication
-

8.19 Third-Party Authentication (Future)

Supported Providers

- Google
- Microsoft
- LinkedIn
- GitHub

Enterprise

- OAuth 2.0
 - OpenID Connect
 - SAML 2.0
-

8.20 Calendar Integration (Future)

Supported

- Google Calendar
- Microsoft Outlook

Use Cases

- Interview Scheduling
 - Reminder Notifications
-

8.21 Video Meeting Integration (Future)

Supported Platforms

- Google Meet
- Microsoft Teams
- Zoom

Features

- Interview Meeting Links
 - Calendar Invitations
 - Automatic Notifications
-

8.22 Payment Integration (Future)

Supported Providers

- Razorpay
- Stripe
- PayPal

Potential Use Cases

- Premium Plans
 - Certification Fees
 - Recruiter Subscription
 - Feature Upgrades
-

8.23 Integration Monitoring

Monitor

- API Response Time
- Success Rate
- Failure Rate
- Retry Count
- Timeout Count
- Third-Party Availability

Alerts

- High Error Rate
 - Slow Response
 - Service Unavailable
-

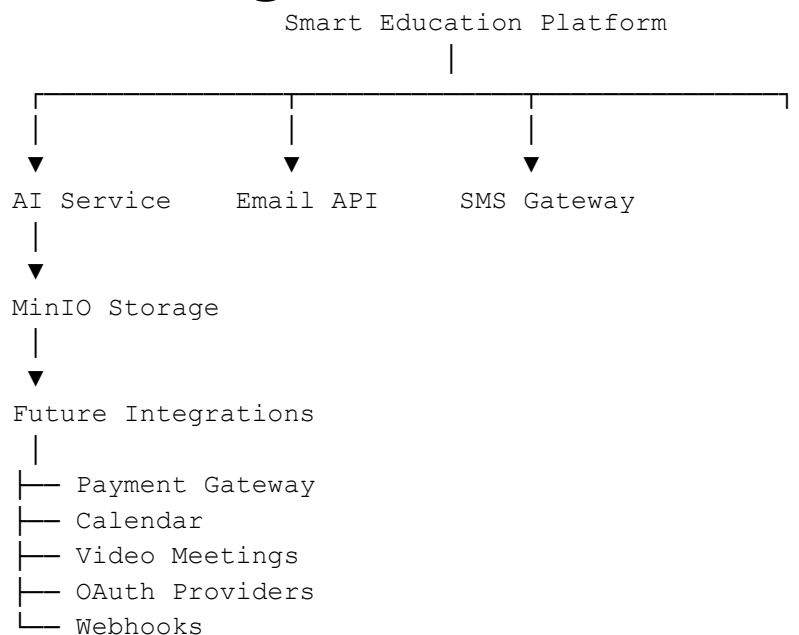
8.24 Integration Logging

Log

- Request ID
- API Endpoint
- Provider
- Response Code
- Latency
- Retry Attempts
- Error Details

Sensitive information such as passwords, API keys, and personal data must never be written to logs.

8.25 Integration Architecture Diagram



8.26 Integration Best Practices

- Keep integrations loosely coupled.
 - Use standardized REST APIs.
 - Secure all external communication with HTTPS.
 - Validate all incoming and outgoing data.
 - Implement retries with exponential backoff.
 - Use circuit breakers for fault tolerance.
 - Log integration failures with correlation IDs.
 - Monitor third-party service health continuously.
 - Version external APIs to support backward compatibility.
 - Design integrations for future event-driven architecture.
-

8.27 Future Integration Roadmap

Phase 1

- AI Provider
- Email
- SMS
- MinIO

Phase 2

- OAuth Providers
- Calendar Integration
- Video Meeting Platforms
- Webhooks
- API Gateway

Phase 3

- Kafka Event Streaming
 - Enterprise ERP
 - HRMS
 - LMS
 - CRM
 - Business Intelligence Platforms
-

8.28 Chapter Summary

This chapter defines the **Integration Architecture** of the Smart Education Platform. It establishes a secure, API-first integration model connecting internal modules with external services such as AI providers, email systems, SMS gateways, and object storage. The architecture incorporates retry mechanisms, circuit breakers, monitoring, logging, and security controls while preparing the platform for future integrations including API gateways, event-driven messaging, payment services, calendar systems, and enterprise identity providers.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 9 – Scalability, Performance & High Availability Architecture

Document Version: 1.0

Architecture Style: Modular Monolith (Phase 1) → Cloud Native Auto-Scaling (Phase 2)

9.1 Introduction

This chapter defines the scalability, performance optimization, and high availability strategy for the **Smart Education Platform**. The architecture ensures that the platform can support increasing numbers of users while maintaining fast response times, high uptime, and reliable service delivery.

The design is optimized for Phase 1 while providing a clear migration path to a distributed cloud-native architecture.

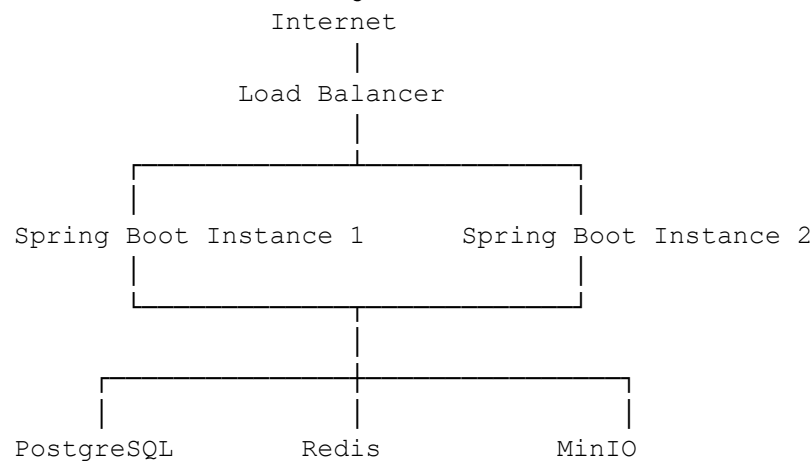
9.2 Objectives

The architecture shall provide:

- High Performance
- Horizontal Scalability
- Vertical Scalability

- High Availability
 - Low Latency
 - Fault Tolerance
 - Elastic Growth
 - Efficient Resource Utilization
 - Business Continuity
-

9.3 Scalability Architecture



The application layer can scale independently by adding additional application instances.

9.4 Performance Goals

Metric	Target
API Response Time	< 300 ms
Login Response	< 2 Seconds
Dashboard Loading	< 2 Seconds
File Upload	< 5 Seconds
AI Response	< 10 Seconds
Database Query	< 100 ms
Cache Response	< 10 ms
Platform Availability	99.9%

9.5 Horizontal Scaling

Horizontal scaling is achieved by adding more application instances.

Users

↓

Load Balancer

↓

Spring Boot Instance 1

Spring Boot Instance 2

Spring Boot Instance 3

Spring Boot Instance N

Benefits

- Increased throughput
 - Improved fault tolerance
 - Better resource utilization
 - Reduced downtime
-

9.6 Vertical Scaling

Resources of existing servers can be increased.

Examples

- CPU
- Memory
- Storage
- Network Bandwidth

Suitable for:

- Early-stage deployments
 - Temporary load increases
-

9.7 Load Balancing

Responsibilities

- Traffic Distribution
- Health Checks
- Failover
- SSL Termination
- Session Routing (if required)

Recommended

- Nginx
 - HAProxy
 - Cloud Load Balancer (AWS ALB, Azure Load Balancer, GCP Load Balancer)
-

9.8 Caching Architecture

Technology

- Redis

Cached Data

- User Sessions
- Dashboard Data
- Leaderboards
- Master Data
- Frequently Accessed Questions
- AI Responses
- Configuration

Benefits

- Reduced database load
 - Faster response time
 - Improved user experience
-

9.9 Database Scalability

Phase 1

- Single PostgreSQL Primary Database

Phase 2

- Read Replicas
- Connection Pooling
- Query Optimization

Phase 3

- Database Partitioning
 - Multi-region Replication
-

9.10 Connection Pooling

Technology

- HikariCP

Benefits

- Faster database access
 - Reduced connection overhead
 - Better resource management
-

9.11 Object Storage Scalability

Technology

- MinIO

Stores

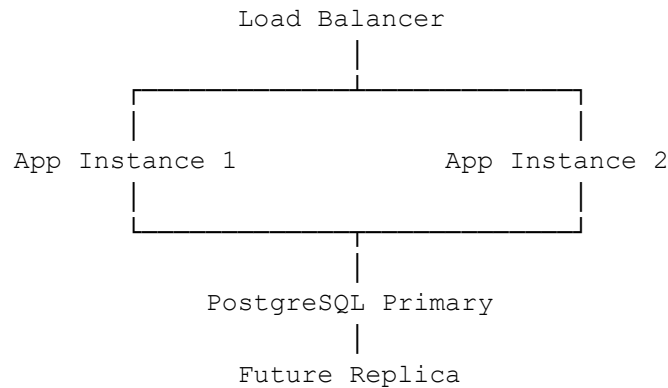
- Resumes
- Images
- Certificates
- Company Logos
- Reports

Future

- Distributed Object Storage
 - Object Versioning
 - Geo-Replication
-

9.12 High Availability (HA)

High availability is achieved through redundancy.



Target Availability

99.9%

9.13 Fault Tolerance

Mechanisms

- Health Checks
 - Automatic Restart
 - Retry Logic
 - Circuit Breaker
 - Graceful Degradation
-

9.14 Capacity Planning

Expected Phase 1 Capacity

Resource	Target
Learners	100,000+

Resource	Target
Recruiters	5,000+
Super Admins	50+
Concurrent Users	5,000+
Daily API Requests	2 Million+
Daily AI Requests	100,000+

9.15 Performance Optimization

Optimization Techniques

- Redis Caching
 - Database Indexing
 - Lazy Loading
 - Pagination
 - Batch Processing
 - Compression (GZIP/Brotli)
 - Asynchronous Processing
 - CDN (Future)
-

9.16 Asynchronous Processing

Suitable Tasks

- Email Delivery
- SMS Delivery
- Report Generation
- AI Processing
- Notification Broadcasting

Future Technologies

- Kafka
 - RabbitMQ
-

9.17 CDN Strategy (Future)

Static Assets

- Images
- CSS
- JavaScript
- Fonts
- Documents

Benefits

- Reduced latency
 - Global availability
 - Lower server load
-

9.18 Monitoring

Monitor

- CPU Usage
- Memory Usage
- Disk Space
- API Latency
- JVM Metrics
- Database Performance
- Redis Usage
- Active Users

Tools

- Prometheus
 - Grafana
-

9.19 Disaster Recovery

Recovery Targets

Metric	Target
--------	--------

RPO	15 Minutes
-----	------------

RTO	1 Hour
-----	--------

Recovery Includes

- Database Restore
 - Object Storage Restore
 - Application Redeployment
 - Configuration Recovery
-

9.20 Auto Scaling (Future)

Kubernetes-based scaling

CPU > 70%

↓

Horizontal Pod Autoscaler

↓

Launch New Pod

↓

Load Balancer Updates

9.21 Multi-Region Deployment (Future)

Region A

↓

Primary Database

↓

Replication

↓

Region B

↓

Disaster Recovery

Benefits

- Geographic redundancy
 - Improved disaster recovery
 - Lower latency for global users
-

9.22 Performance Testing

Testing Types

- Load Testing
- Stress Testing
- Spike Testing
- Endurance Testing
- Volume Testing

Recommended Tools

- Apache JMeter
 - Gatling
 - k6
-

9.23 Production Readiness Checklist

Before Go-Live

- Performance testing completed
 - Database indexed
 - Redis configured
 - Monitoring enabled
 - Logging configured
 - SSL installed
 - Backup verified
 - Health checks active
 - Alerts configured
-

9.24 Architecture Evolution

Phase 1

- Modular Monolith
- Docker
- Single Database
- Redis Cache

Phase 2

- Kubernetes
- Read Replicas
- API Gateway
- Kafka

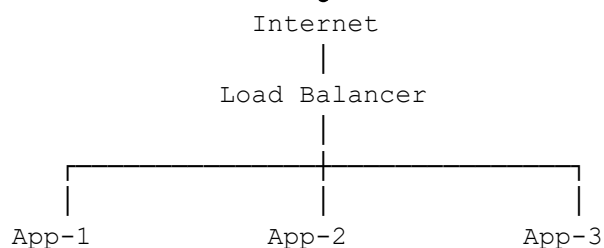
Phase 3

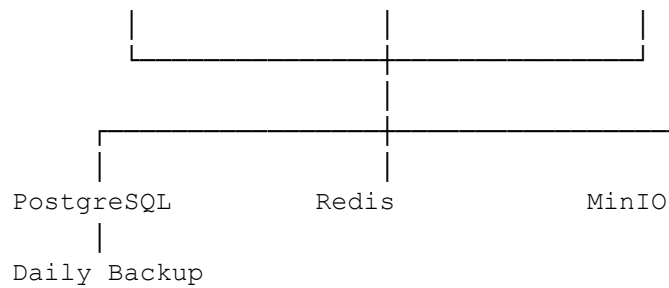
- Microservices
 - Multi-Region Deployment
 - Auto Scaling
 - Service Mesh
-

9.25 Best Practices

- Cache frequently accessed data.
 - Use pagination for large datasets.
 - Optimize SQL queries and indexes.
 - Minimize synchronous processing.
 - Monitor application performance continuously.
 - Implement graceful degradation during failures.
 - Test scalability regularly under production-like loads.
 - Design services to remain stateless where practical.
-

9.26 Scalability Architecture Diagram





9.27 Chapter Summary

This chapter defines the **Scalability, Performance, and High Availability Architecture** of the Smart Education Platform. It outlines strategies for horizontal and vertical scaling, load balancing, Redis-based caching, database optimization, connection pooling, fault tolerance, disaster recovery, and production monitoring. The architecture targets **99.9% availability**, supports **100,000+ learners**, and establishes a clear roadmap from a modular monolith to a highly scalable cloud-native platform with Kubernetes, auto-scaling, and multi-region deployment.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 10 – DevOps, CI/CD & Release Management Architecture

Document Version: 1.0

Architecture Style: GitOps Ready • Containerized Deployment • CI/CD Driven

10.1 Introduction

The **DevOps, CI/CD, and Release Management Architecture** defines the strategy for building, testing, deploying, monitoring, and maintaining the Smart Education Platform throughout its lifecycle.

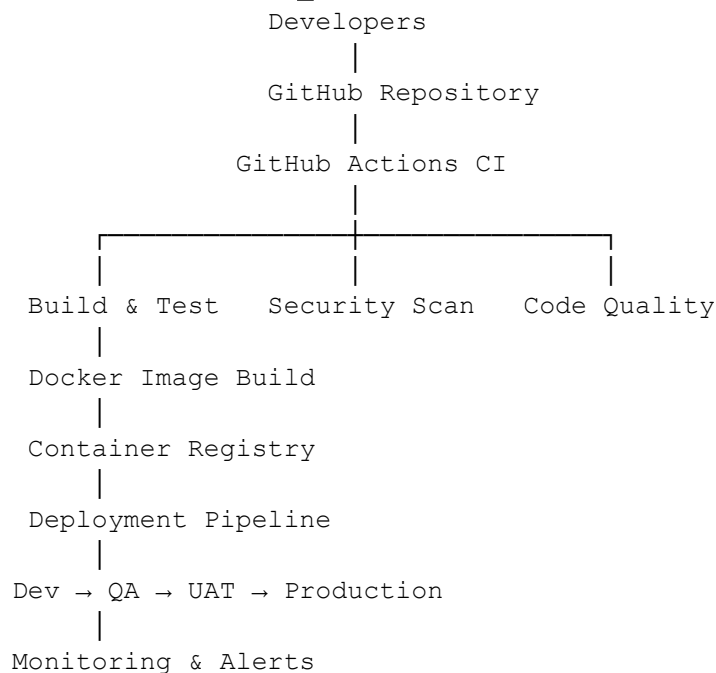
The objective is to deliver software rapidly, reliably, and securely with minimal downtime while maintaining high quality and repeatable deployments.

10.2 DevOps Objectives

The DevOps architecture shall:

- Automate build and deployment
 - Reduce deployment failures
 - Enable continuous integration
 - Enable continuous delivery
 - Ensure zero/minimal downtime
 - Improve rollback capability
 - Standardize environments
 - Support Infrastructure as Code (IaC)
 - Increase deployment frequency
 - Maintain production stability
-

10.3 DevOps Architecture Overview



10.4 DevOps Toolchain

Category	Technology
----------	------------

Category	Technology
Version Control	Git + GitHub
CI/CD	GitHub Actions
Build Tool	Maven
Backend	Spring Boot
Frontend	React
Mobile	Flutter
Containerization	Docker
Reverse Proxy	Nginx
Database	PostgreSQL
Cache	Redis
Object Storage	MinIO
Monitoring	Prometheus + Grafana
Logging	Logback (ELK in Future)

10.5 Git Branching Strategy

The platform follows a **GitFlow** model.

```

main
|
|— production
|
|— develop
|
|— feature/*
|
|— release/*
|
|— hotfix/*

```

Branch Purpose

- **main** – Stable production-ready code
 - **develop** – Integration branch
 - **feature/** – New features
 - **release/** – Release preparation
 - **hotfix/** – Critical production fixes
-

10.6 Development Workflow

Requirement

↓

Feature Branch

↓

Development

↓

Pull Request

↓

Code Review

↓

Merge to Develop

↓

QA Testing

↓

Release Branch

↓

Production

10.7 Continuous Integration (CI)

Every commit automatically triggers:

- Source Code Checkout
- Dependency Download
- Maven Build
- Unit Tests
- Static Code Analysis

- Security Scan
 - Docker Image Build
 - Artifact Generation
-

10.8 Continuous Delivery (CD)

Deployment pipeline:

GitHub Actions

↓

Build

↓

Docker Image

↓

QA Deployment

↓

UAT Deployment

↓

Production Deployment

Deployment is automated after successful validation.

10.9 Build Pipeline

Pipeline Steps

1. Checkout Source
2. Install Dependencies
3. Compile Project
4. Execute Unit Tests
5. Generate Build Artifact
6. Build Docker Image
7. Push Image to Registry

8. Deploy Environment
 9. Run Smoke Tests
 10. Notify Team
-

10.10 Docker Build Architecture

Source Code

↓

Docker Build

↓

Application Image

↓

Docker Registry

↓

Deployment Server

Separate Docker images for:

- Backend
 - Frontend
 - Monitoring Tools
-

10.11 Environment Promotion Strategy

Developer

↓

Development

↓

QA

↓

UAT

↓

Production

Each environment has:

- Separate database
 - Separate Redis
 - Separate MinIO
 - Separate configuration
 - Independent secrets
-

10.12 Infrastructure as Code (IaC)

Infrastructure should be version-controlled.

Future Technologies

- Terraform
- Ansible
- Kubernetes Manifests
- Helm Charts

Benefits

- Repeatable deployments
 - Reduced configuration drift
 - Faster provisioning
-

10.13 Secrets Management

Sensitive information includes:

- Database Passwords
- JWT Secret
- AI API Keys
- SMTP Credentials
- Redis Password

Best Practices

- Environment Variables
 - Secret Vault (Future)
 - Encrypted Storage
 - Never commit secrets to Git
-

10.14 Deployment Strategies

Supported Strategies

Phase 1

- Rolling Deployment

Future

- Blue-Green Deployment
- Canary Deployment

Benefits

- Reduced downtime
 - Easy rollback
 - Safer releases
-

10.15 Release Management

Release Types

- Major Release
- Minor Release
- Patch Release
- Hotfix

Version Format

v1.0.0

v1.1.0

v1.1.1

10.16 Rollback Strategy

Rollback Triggers

- Critical Bug
- High Error Rate
- Deployment Failure
- Performance Degradation

Rollback Process

Deploy Previous Version

↓

Restart Services

↓

Health Verification

↓

Notify Team

10.17 Quality Gates

Deployment proceeds only if:

- Build Successful
 - Unit Tests Passed
 - Code Coverage Threshold Met
 - Security Scan Passed
 - Smoke Tests Passed
 - Manual Approval (Production)
-

10.18 Automated Testing Pipeline

Testing includes:

- Unit Testing
- Integration Testing
- API Testing
- UI Testing
- Smoke Testing
- Regression Testing

Future

- Performance Testing
- Security Penetration Testing

10.19 Production Deployment Flow

Developer

↓

Git Push

↓

GitHub Actions

↓

Docker Build

↓

Deploy

↓

Health Check

↓

Monitoring

↓

Production Ready

10.20 Monitoring After Deployment

Monitor

- API Health
- CPU Usage
- Memory Usage
- JVM Metrics
- Database Performance
- Redis Performance
- Disk Usage
- Error Rate

Alerts

- High CPU
 - Slow Response
 - Failed Health Checks
 - Service Downtime
-

10.21 Incident Management

Incident Lifecycle

Issue Detected

↓

Alert

↓

Investigation

↓

Fix

↓

Deployment

↓

Verification

↓

Closure

10.22 Backup Before Release

Before every production deployment:

- Database Backup
- MinIO Backup
- Configuration Backup
- Docker Image Backup

This ensures safe rollback if required.

10.23 Release Checklist

Before production deployment verify:

- Code Review Completed
 - Build Successful
 - Tests Passed
 - Database Migration Verified
 - Docker Images Available
 - SSL Valid
 - Monitoring Active
 - Backups Completed
 - Rollback Plan Ready
 - Stakeholder Approval Received
-

10.24 DevOps Metrics

Track the following KPIs:

Metric	Target
Deployment Frequency	Daily/Weekly

Metric	Target
Build Success Rate	> 95%
Mean Time to Recovery (MTTR)	< 1 Hour
Change Failure Rate	< 5%
Deployment Duration	< 15 Minutes
Production Availability	99.9%

10.25 Production Architecture

GitHub

↓

GitHub Actions

↓

Docker Registry

↓

Production Server

↓

Nginx

↓

Spring Boot

↓

PostgreSQL

↓

Redis

↓

MinIO

↓

10.26 Future DevOps Roadmap

Phase 1

- Docker
- GitHub Actions
- Rolling Deployment
- Manual Production Approval

Phase 2

- Kubernetes
- Helm
- Terraform
- Blue-Green Deployment
- Automated Scaling

Phase 3

- GitOps
 - ArgoCD
 - Service Mesh
 - Multi-Region Deployment
 - Self-Healing Infrastructure
-

10.27 Best Practices

- Keep deployments automated and repeatable.
- Use pull requests with mandatory code reviews.
- Maintain separate environments for each deployment stage.
- Never store secrets in source code.
- Tag every production release.
- Verify health checks after deployment.
- Maintain comprehensive deployment logs.
- Test rollback procedures regularly.
- Automate infrastructure provisioning where practical.

- Continuously monitor production systems.
-

10.28 Chapter Summary

This chapter defines the **DevOps, CI/CD, and Release Management Architecture** for the Smart Education Platform. It establishes a standardized development workflow using **GitHub**, **GitHub Actions**, **Docker**, and **Spring Boot**, supported by automated builds, testing, containerized deployments, environment promotion, monitoring, rollback procedures, and release governance. The architecture ensures reliable, secure, and repeatable software delivery while providing a clear evolution path toward Kubernetes, GitOps, and fully automated cloud-native operations.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 11 – Monitoring, Observability & Logging Architecture

Document Version: 1.0

Architecture Style: Enterprise Observability with Metrics, Logs & Health Monitoring

11.1 Introduction

Monitoring and observability ensure that the **Smart Education Platform** remains reliable, secure, and performant in production. This architecture provides real-time visibility into application health, infrastructure status, business operations, and security events.

The platform adopts the **Three Pillars of Observability**:

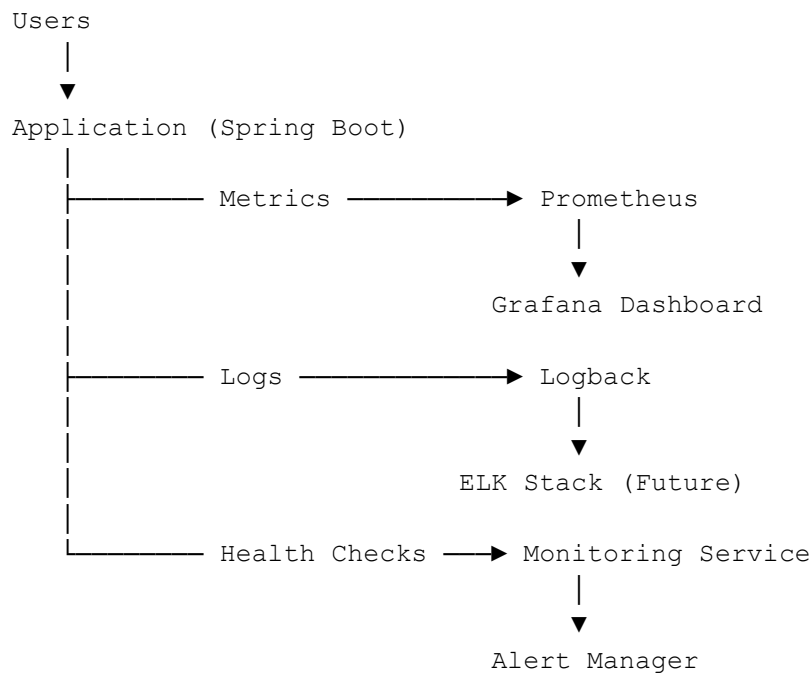
- Metrics
 - Logs
 - Traces (Future Phase)
-

11.2 Objectives

The monitoring architecture shall provide:

- Real-time monitoring
 - Complete observability
 - Proactive alerting
 - Centralized logging
 - Performance analysis
 - Root cause identification
 - Capacity monitoring
 - Security monitoring
 - Business monitoring
-

11.3 Overall Monitoring Architecture



11.4 Observability Pillars

Metrics

Measure system performance.

Examples

- CPU
 - Memory
 - API Response Time
 - JVM Usage
 - Database Connections
-

Logs

Record application events.

Examples

- Login
 - Exceptions
 - Database Errors
 - API Calls
 - Security Events
-

Traces (Future)

Track requests across multiple services.

Future Technology

- OpenTelemetry
 - Jaeger
 - Zipkin
-

11.5 Application Monitoring

Monitor

- API Response Time
- Active Users
- Authentication Success
- Authentication Failure
- Error Rate
- Background Jobs
- Cache Usage

- AI Requests
-

11.6 Infrastructure Monitoring

Monitor

- CPU
- RAM
- Disk Usage
- Network
- Docker Containers
- Database
- Redis
- MinIO

Threshold Example

Metric Warning Critical

CPU	70%	90%
RAM	75%	90%
Disk	80%	95%

11.7 Database Monitoring

Monitor

- Active Connections
- Slow Queries
- Query Execution Time
- Locks
- Replication Status (Future)
- Transaction Rate
- Database Size

Alerts

- Slow Query
- Connection Exhaustion
- High Disk Usage

11.8 Redis Monitoring

Monitor

- Memory Usage
 - Cache Hit Ratio
 - Cache Miss Ratio
 - Connected Clients
 - Expired Keys
 - Evicted Keys
-

11.9 MinIO Monitoring

Monitor

- Storage Capacity
 - Bucket Usage
 - Upload Errors
 - Download Errors
 - Object Count
 - Request Latency
-

11.10 Business Monitoring

Monitor

- Daily Registrations
 - Active Learners
 - Active Recruiters
 - Job Applications
 - Coding Submissions
 - AI Requests
 - Resume Downloads
 - Placement Success Rate
-

11.11 API Monitoring

Track

- Total Requests
- Success Rate
- Failure Rate
- Latency
- Endpoint Usage
- HTTP Status Codes

Example Metrics

GET /api/jobs

Average Response = 120 ms

Success Rate = 99.7%

11.12 Logging Architecture

Spring Boot

↓

Logback

↓

Log Files

↓

ELK Stack (Future)

↓

Dashboard

Log Levels

- TRACE
- DEBUG
- INFO
- WARN

- ERROR
-

11.13 Log Categories

Application Logs

- Startup
- Shutdown
- Configuration

Business Logs

- Job Published
- Resume Updated
- Challenge Completed

Security Logs

- Login
- Logout
- Failed Login
- Permission Denied

Audit Logs

- Admin Actions
- Recruiter Actions
- User Changes

System Logs

- JVM
 - Docker
 - Database
-

11.14 Structured Logging

Each log entry should include

- Timestamp

- Request ID
 - User ID
 - Role
 - Module
 - API
 - Status Code
 - Execution Time
 - IP Address
-

11.15 Health Check Architecture

Endpoints

`/actuator/health`

`/actuator/info`

`/actuator/metrics`

`/actuator/prometheus`

Checks

- Database
 - Redis
 - MinIO
 - Disk Space
 - Memory
 - External APIs
-

11.16 Metrics Collection

Technology

- Micrometer
- Prometheus

Collected Metrics

- JVM
- Heap Memory

- Threads
 - GC
 - API
 - Database
 - Redis
 - Cache
-

11.17 Grafana Dashboards

Dashboards

Infrastructure

- CPU
- RAM
- Disk

Application

- API Response
- Error Rate
- JVM

Business

- Users
- Jobs
- Placement
- AI Usage

Security

- Login Attempts
 - Failed Logins
 - JWT Validation Errors
-

11.18 Alert Management

Alert Channels

- Email
- SMS
- Slack (Future)
- Microsoft Teams (Future)

Critical Alerts

- Server Down
 - Database Down
 - Redis Down
 - SSL Expiry
 - High Error Rate
 - AI Service Failure
-

11.19 Audit Trail

Capture

- Login
- Logout
- User Update
- Recruiter Approval
- Question Creation
- Job Posting
- Resume Download
- Configuration Changes

Fields

- User
 - Role
 - Action
 - Timestamp
 - IP Address
 - Status
-

11.20 Security Monitoring

Monitor

- Brute Force Attempts
 - Token Misuse
 - Unauthorized Access
 - SQL Injection Attempts
 - XSS Attempts
 - Rate Limit Violations
-

11.21 Incident Detection

Problem

↓

Monitoring

↓

Alert

↓

Investigation

↓

Root Cause

↓

Resolution

↓

Postmortem

11.22 Root Cause Analysis (RCA)

Collect

- Logs
- Metrics
- Database Status
- API Traces (Future)

- Infrastructure Events

Output

- Incident Report
 - Corrective Actions
 - Preventive Actions
-

11.23 SRE Practices

Objectives

- Service Reliability
- Error Budget
- SLA Monitoring
- SLO Tracking
- Capacity Planning

Example SLOs

Service	Target
API Availability	99.9%
Login Success	99.5%
Dashboard Load	<2 sec
AI Response	<10 sec

11.24 Operational Dashboards

Operations Team

- Server Health
- Deployments
- Alerts

Support Team

- Active Users
- Open Issues
- Error Trends

Management

- Growth
 - Placements
 - Recruiter Activity
 - Platform KPIs
-

11.25 Log Retention Policy

Log Type	Retention
Application Logs	90 Days
Security Logs	1 Year
Audit Logs	7 Years
Access Logs	180 Days
Error Logs	180 Days

11.26 Future Observability Roadmap

Phase 2

- OpenTelemetry
- Jaeger
- Zipkin
- Distributed Tracing
- ELK Stack
- AI-based Anomaly Detection

Phase 3

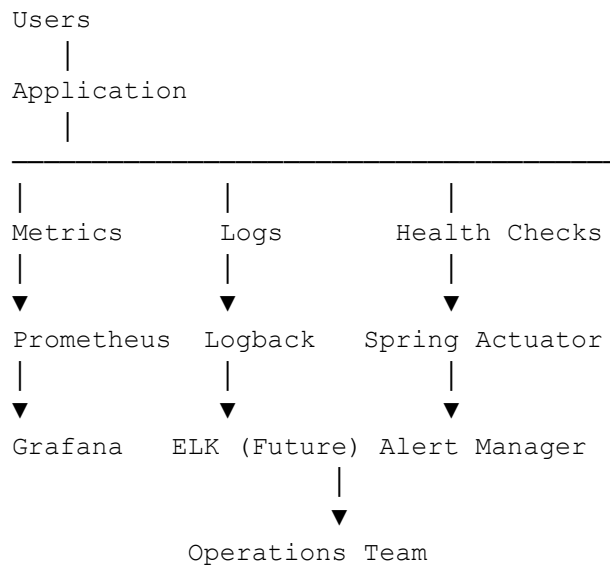
- Predictive Monitoring
 - Automated Incident Resolution
 - Self-Healing Infrastructure
 - ML-based Capacity Planning
-

11.27 Best Practices

- Monitor every critical service.

- Use structured JSON logging.
 - Configure actionable alerts to reduce alert fatigue.
 - Protect logs from unauthorized access.
 - Monitor both technical and business KPIs.
 - Regularly review dashboards and thresholds.
 - Archive logs according to retention policies.
 - Perform periodic disaster recovery and monitoring drills.
-

11.28 Monitoring & Observability Architecture



11.29 Chapter Summary

This chapter defines the **Monitoring, Observability & Logging Architecture** for the Smart Education Platform. It establishes a production-ready monitoring strategy using **Spring Boot Actuator**, **Micrometer**, **Prometheus**, **Grafana**, and **Logback**, with future support for **OpenTelemetry** and the **ELK Stack**. The architecture enables real-time health monitoring, centralized logging, proactive alerting, business KPI tracking, audit trails, and security monitoring, ensuring operational excellence and rapid incident response.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 12 – Business Process Architecture & End-to-End Workflow Design

Document Version: 1.0

Architecture Style: Event-Driven Business Process with Modular Workflow

12.1 Introduction

This chapter defines the **end-to-end business processes** of the Smart Education Platform. It describes how users interact with the platform, how different modules collaborate, and how business events flow across the system.

The workflow architecture ensures:

- Standardized business processes
 - Modular execution
 - Consistent user experience
 - Complete auditability
 - Future automation readiness
-

12.2 Business Process Objectives

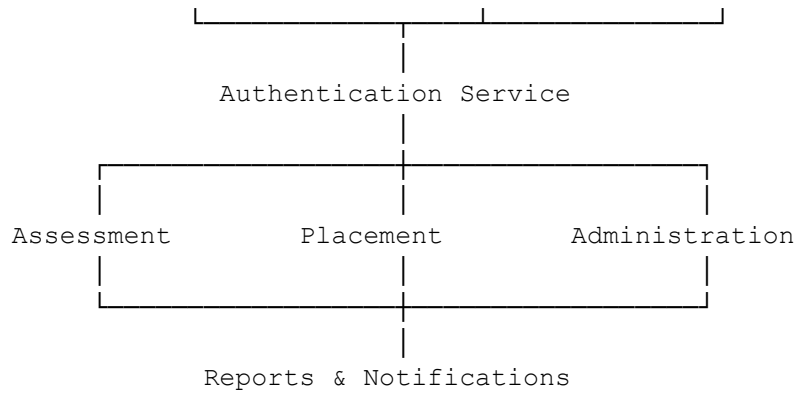
The platform shall:

- Simplify learner journeys
 - Automate recruitment workflows
 - Improve administrative efficiency
 - Support event-driven processing
 - Provide complete traceability
 - Reduce manual intervention
-

12.3 End-to-End Business Architecture

Smart Education Platform

Independent Learner	Recruiter	Super Admin



12.4 Independent Learner Journey

Registration

↓

Login

↓

Profile Completion

↓

Dashboard

↓

Brain Gym

↓

Aptitude Arena

↓

Coding Arena

↓

SQL Arena

↓

Debug Hero

↓

AI Coach

↓

Resume Builder

↓

Placement Portal

↓

Apply Jobs

↓

Interview

↓

Offer

12.5 Learner Registration Workflow

Open App

↓

Register

↓

Email Verification

↓

Account Activation

↓

Login

↓

Complete Profile

↓

Dashboard Ready

Business Rules

- Email must be unique
 - Password policy enforced
 - Email verification required
 - Profile completion recommended before job applications
-

12.6 Authentication Workflow

Login Request

↓

Credential Validation

↓

JWT Generation

↓

Role Validation

↓

Dashboard Access

Security Controls

- JWT Authentication
 - Refresh Token
 - Account Lock (Future)
 - Audit Logging
-

12.7 Dashboard Workflow

Dashboard displays:

- XP
- Coins
- Level
- Daily Challenges
- Placement Score
- Recent Activity
- AI Suggestions
- Notifications

Dashboard data is assembled from multiple modules before rendering.

12.8 Brain Gym Workflow

Select Challenge

↓

Load Questions

↓

Solve Challenge

↓

Submit

↓

Evaluation

↓

XP Award

↓

Leaderboard Update

↓

Dashboard Refresh

12.9 Aptitude Arena Workflow

Choose Test

↓

Start Timer

↓

Answer Questions

↓

Submit Test

↓

Evaluation

↓

Ranking

↓

Reports

↓

XP Update

12.10 Coding Arena Workflow

Choose Problem

↓

Write Code

↓

Compile

↓

Execute Test Cases

↓

Submit

↓

Evaluation

↓

Score

↓

Leaderboard

Events Generated

- Coding Completed
 - XP Earned
 - Report Updated
-

12.11 SQL Arena Workflow

Choose SQL Challenge

↓

Write Query

↓

Execute

↓

Validate Result

↓

Score

↓

XP

↓

Reports

12.12 Debug Hero Workflow

Choose Bug

↓

Analyze Code

↓

Fix Error

↓

Submit

↓

Evaluation

↓

Leaderboard

↓

Reward

12.13 Gamification Workflow

Activity Completed

↓

XP Calculation

↓

Coins Awarded

↓

Badge Validation

↓

Level Check

↓

Leaderboard Update

↓

Notification

Possible Rewards

- XP
 - Coins
 - Badges
 - Achievement Unlocks
-

12.14 AI Coach Workflow

Ask Question

↓

Context Collection

↓

Prompt Generation

↓

AI Processing

↓

Response Validation

↓

Formatted Response

↓

Conversation Saved

Use Cases

- Resume Review
- Career Guidance
- Interview Tips
- Coding Help
- SQL Help

12.15 Resume Builder Workflow

Create Resume

↓

Enter Details

↓

Generate Resume

↓

AI Review

↓

ATS Score

↓

Download PDF

↓

Apply for Jobs

12.16 Placement Workflow

Search Jobs

↓

View Job

↓

Apply

↓

Application Submitted

↓

Recruiter Review

↓

Shortlisted

↓

Interview

↓

Offer

↓

Placement Successful

Application Status

- Applied
- Under Review
- Shortlisted
- Interview Scheduled
- Selected
- Offer Released
- Rejected

12.17 Recruiter Workflow

Recruiter Login

↓

Dashboard

↓

Create Job

↓

Publish Job

↓

Receive Applications

↓

Screen Resume

↓

Shortlist

↓

Schedule Interview

↓

Generate Offer

12.18 Super Admin Workflow

Login

↓

Dashboard

↓

Manage Users

↓

Approve Recruiters

↓

Manage Questions

↓

Configure Gamification

↓

Monitor Platform

↓

Reports

↓

Audit Logs

12.19 Notification Workflow

Business Events

- Registration
- Password Reset
- Job Posted
- Job Applied
- Interview Scheduled
- Offer Released
- Badge Earned

Flow

Business Event

↓

Notification Service

↓

Email

↓

SMS

↓

In-App Notification

12.20 Reporting Workflow

Business Events

↓

Database

↓

Analytics Engine

↓

Dashboard

↓

PDF/Excel Export

Reports

- Learner
 - Recruiter
 - Admin
 - Placement
 - Gamification
-

12.21 Business Event Architecture

Major Events

- User Registered
- Login Successful
- Assessment Completed
- Badge Earned
- Resume Updated

- Job Applied
- Interview Scheduled
- Offer Released
- Recruiter Approved

Each event updates:

- Reports
 - Notifications
 - Audit Logs
 - Analytics
-

12.22 Cross-Module Communication

Authentication

↓

Assessment Modules

↓

Gamification

↓

AI Coach

↓

Resume Builder

↓

Placement Portal

↓

Recruiter Portal

↓

Reports

↓

Notifications

Each module communicates through service interfaces, avoiding direct dependencies.

12.23 End-to-End Placement Workflow

Learner

↓

Practice

↓

Improve Score

↓

Create Resume

↓

Apply Job

↓

Recruiter Review

↓

Interview

↓

Offer

↓

Placement

12.24 Error Handling Workflow

Request

↓

Validation

↓

Business Processing

↓

Success

OR

Exception

↓

Global Error Handler

↓

User-Friendly Response

↓

Audit Log

12.25 Workflow Monitoring

Track

- Active Users
 - Running Assessments
 - Job Applications
 - Interview Status
 - AI Requests
 - Notification Delivery
 - System Errors
-

12.26 Future Workflow Enhancements

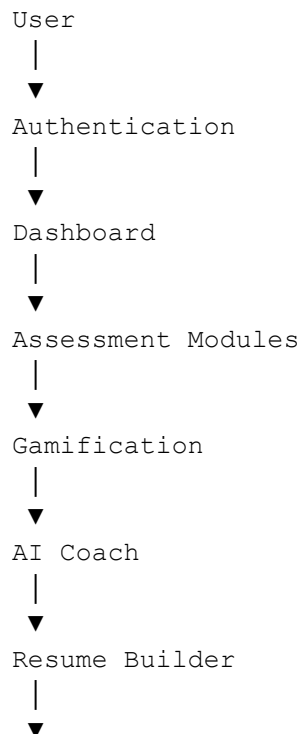
Phase 2

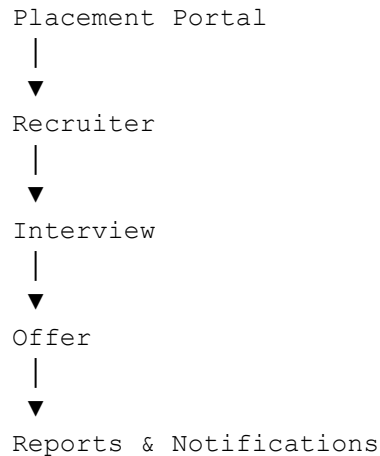
- Learning Module
 - Live Classes
 - Mentor Interaction
 - Parent Portal
 - Workflow Automation
 - Event Streaming (Kafka)
 - AI Recommendation Engine
 - Multi-Tenant Business Processes
-

12.27 Business Process Best Practices

- Keep workflows simple and modular.
 - Validate data at every step.
 - Record audit logs for critical actions.
 - Use asynchronous processing for notifications and reports.
 - Ensure workflows are resilient to failures.
 - Minimize manual intervention through automation.
 - Design processes for future scalability.
-

12.28 End-to-End Workflow Architecture





12.29 Chapter Summary

This chapter defines the **Business Process Architecture & End-to-End Workflow Design** for the Smart Education Platform. It describes the complete lifecycle of learners, recruiters, and administrators—from registration and authentication to assessments, gamification, AI assistance, resume building, job applications, recruitment, reporting, and notifications. By structuring these workflows into modular, event-driven processes, the architecture ensures consistency, scalability, auditability, and readiness for future enhancements such as workflow automation, event streaming, and advanced AI-driven personalization.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 13 – Network Architecture & Communication Design

Document Version: 1.0

Architecture Style: Secure Layered Network Architecture with Zero Trust Communication

13.1 Introduction

The **Network Architecture** defines how users, applications, infrastructure components, databases, and external services communicate securely within the Smart Education Platform.

The architecture is designed to provide:

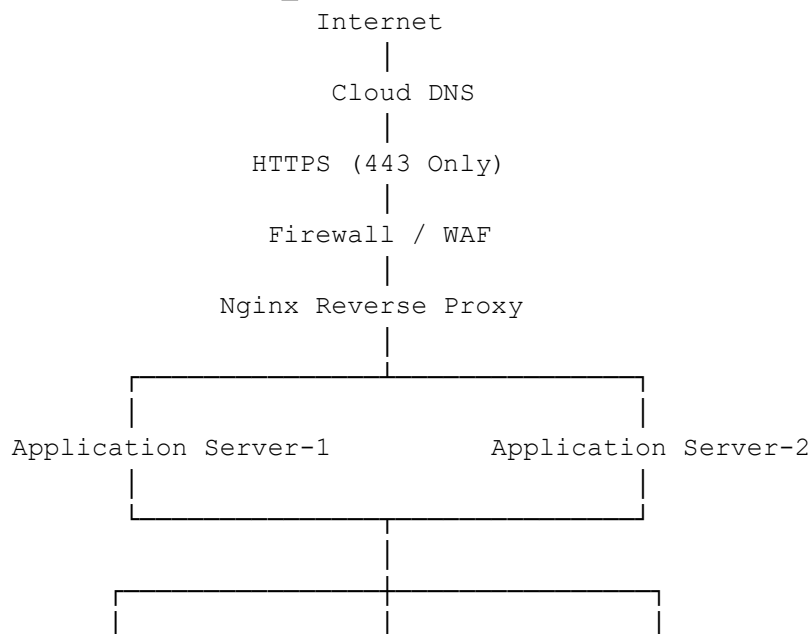
- Secure communication
 - High availability
 - Low latency
 - Fault tolerance
 - Network isolation
 - Future cloud-native networking
-

13.2 Objectives

The network architecture shall provide:

- Secure communication
 - Network segmentation
 - HTTPS-only communication
 - High availability
 - Secure administrative access
 - Scalable infrastructure
 - Disaster recovery support
 - Future Kubernetes compatibility
-

13.3 Enterprise Network Architecture



PostgreSQL
|
Backup Server

Redis

MinIO Storage

13.4 Network Layers

The network is divided into logical layers.

Internet

↓

Edge Layer

↓

Reverse Proxy Layer

↓

Application Layer

↓

Data Layer

↓

Storage Layer

↓

Monitoring Layer

Each layer has dedicated responsibilities and security controls.

13.5 Internet Communication

Users access the platform using:

- Web Browser
- Android App
- iOS App (Future)

Protocols

- HTTPS
- TLS 1.3

Allowed Ports

Port	Purpose
443	HTTPS
80	HTTP → Redirect to HTTPS

13.6 DNS Architecture

DNS Responsibilities

- Domain Resolution
- SSL Validation
- Load Balancer Routing

Example

student.platform.com

↓

DNS

↓

Load Balancer

↓

Application

13.7 Firewall Architecture

Firewall Rules

Allow

- HTTPS (443)

Deny

- Direct Database Access
- Direct Redis Access
- Direct MinIO Access
- Unauthorized SSH

Administrative access is restricted to approved IPs or VPN.

13.8 Reverse Proxy Architecture

Technology

- Nginx

Responsibilities

- SSL Termination
 - Load Balancing
 - Request Routing
 - Compression
 - Security Headers
 - Rate Limiting
 - Static File Delivery
-

13.9 Internal Communication

Application servers communicate internally with:

- PostgreSQL
- Redis
- MinIO
- Monitoring Services

Internal communication occurs over private network segments and is not exposed publicly.

13.10 API Communication

Communication Standard

Client

↓

HTTPS

↓

REST API

↓

JSON

↓

Spring Boot

API Principles

- Stateless
- Secure
- Versioned
- JWT Protected

13.11 Database Communication

Application

↓

Connection Pool

↓

PostgreSQL

Security

- Private Network
 - Authentication
 - TLS (where supported)
 - Connection Pooling
-

13.12 Redis Communication

Application

↓

Redis

Uses

- Session Cache
- OTP
- Leaderboards
- Dashboard Cache

Redis is accessible only from the application network.

13.13 Object Storage Communication

Application

↓

MinIO

↓

Stored Files

Files

- Resume
- Certificates
- Images
- Reports

Only the application server communicates with MinIO.

13.14 AI Communication

Application

↓

HTTPS

↓

AI Provider

↓

Response

Security

- HTTPS
 - API Key
 - Request Validation
 - Timeout
 - Retry Policy
-

13.15 Email Communication

Notification Service

↓

SMTP/API

↓

Email Provider

↓

User

Used For

- Verification
 - Password Reset
 - Interview Notifications
 - Offers
-

13.16 SMS Communication

Notification Service

↓

SMS Gateway

↓

User Mobile

Uses

- OTP
- Alerts
- Interview Reminders

13.17 Administrative Access

Only authorized administrators may access servers.

Methods

- SSH Key Authentication
- VPN (Recommended)
- Bastion Host (Future)

Direct root login should be disabled.

13.18 Network Security Zones

Internet Zone

↓

DMZ

↓

Application Zone

↓

Database Zone

↓

Storage Zone

↓

Monitoring Zone

Each zone communicates only with authorized adjacent zones.

13.19 Network Isolation

Separate logical networks for:

- Application Servers
- Database Servers
- Cache Servers
- Storage Servers
- Monitoring Tools

No direct communication from the Internet to backend services.

13.20 Communication Security

Every communication must support:

- HTTPS
- TLS 1.3
- JWT Authentication
- Secure Headers
- Input Validation

Future

- Mutual TLS (mTLS)
 - Service Identity
-

13.21 Network Monitoring

Monitor

- Bandwidth Usage
- Packet Loss
- Latency
- Connection Count
- Failed Connections
- Firewall Events

Tools

- Prometheus
- Grafana
- Node Exporter

13.22 High Availability Network

Internet

↓

Load Balancer

↓

Application-1

Application-2

↓

Shared Database

↓

Shared Storage

If one application instance fails, traffic is routed automatically to healthy instances.

13.23 Disaster Recovery Network

Primary Site

↓

Database Backup

↓

Secondary Site (Future)

↓

Recovery

The disaster recovery architecture supports restoration of services within the defined RTO and RPO.

13.24 Future Kubernetes Networking

Internet

↓

Ingress Controller

↓

Kubernetes Cluster

↓

Pods

↓

Services

↓

Persistent Storage

Future Enhancements

- Ingress Controller
- Service Discovery
- Internal DNS
- Horizontal Scaling

13.25 Future Service Mesh

Future Technology

- Istio
- Linkerd

Features

- Mutual TLS (mTLS)
 - Traffic Routing
 - Service Discovery
 - Circuit Breaking
 - Distributed Tracing
-

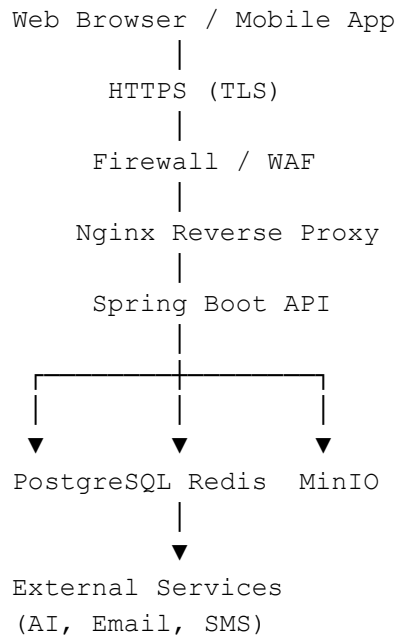
13.26 Network Performance Targets

Metric	Target
API Latency	< 300 ms
Internal Network Latency	< 5 ms
Database Connectivity	> 99.9%
Availability	99.9%
Packet Loss	< 1%

13.27 Network Best Practices

- Expose only HTTPS endpoints to the Internet.
 - Place databases, Redis, and MinIO on private networks.
 - Restrict administrative access through VPN or approved IP addresses.
 - Apply network segmentation between infrastructure layers.
 - Enable continuous network monitoring and alerting.
 - Use secure protocols for all communications.
 - Regularly review firewall rules and network access policies.
 - Prepare the network architecture for future Kubernetes deployment.
-

13.28 Communication Architecture



13.29 Chapter Summary

This chapter defines the **Network Architecture & Communication Design** for the Smart Education Platform. It establishes a secure, layered network model featuring **HTTPS-only communication, network segmentation, private backend services, Nginx reverse proxy, firewall protection, and high availability**. The architecture secures communication between clients, application servers, databases, caches, object storage, and external integrations while providing a clear roadmap for future enhancements such as **Kubernetes networking, service mesh, mutual TLS (mTLS), and multi-site disaster recovery**.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 14 – Disaster Recovery, Backup & Business Continuity Architecture

Document Version: 1.0

Architecture Style: High Availability with Disaster Recovery & Business Continuity

14.1 Introduction

The **Disaster Recovery (DR), Backup, and Business Continuity Architecture** ensures that the Smart Education Platform can continue operating or recover quickly from hardware failures, software failures, cyberattacks, natural disasters, accidental data loss, and operational incidents.

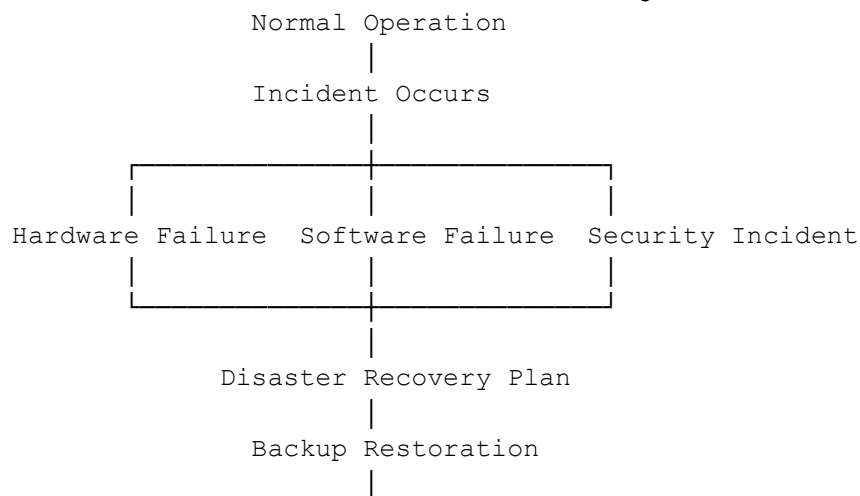
The objective is to minimize downtime, prevent data loss, and maintain uninterrupted business operations.

14.2 Objectives

The architecture shall provide:

- Business Continuity
 - Disaster Recovery
 - Automated Backup
 - Secure Restore Process
 - High Availability
 - Fault Tolerance
 - Operational Resilience
 - Data Integrity
 - Fast Recovery
-

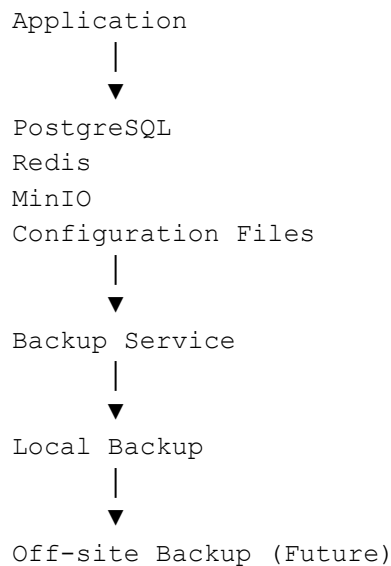
14.3 Business Continuity Architecture



14.4 Disaster Recovery Objectives

Metric	Target
Availability	99.9%
Recovery Point Objective (RPO)	15 Minutes
Recovery Time Objective (RTO)	1 Hour
Backup Success Rate	>99%

14.5 Backup Architecture



14.6 Backup Components

Backups include:

- PostgreSQL Database
- Redis Snapshots
- MinIO Object Storage
- Docker Compose Files
- Environment Configuration
- SSL Certificates
- Application Logs

- Monitoring Configuration
-

14.7 Database Backup Strategy

Incremental Backup

- Every 6 Hours

Full Backup

- Daily

Archive Backup

- Weekly

Backup Verification

- Automatic checksum validation
 - Restore verification
-

14.8 Object Storage Backup

Backup Items

- Resume PDFs
- Certificates
- Profile Images
- Company Logos
- Reports

Storage

- Local Backup Storage
 - Future Cloud Object Storage
-

14.9 Redis Backup

Backup Includes

- Redis Snapshots (RDB)
- Redis Configuration

Redis cache can be rebuilt where appropriate, but critical session and configuration data should be protected.

14.10 Configuration Backup

Backup

- application.yml
- Docker Compose
- Nginx Configuration
- Environment Variables
- SSL Certificates
- Monitoring Configuration

These files are version-controlled where appropriate and backed up securely.

14.11 Disaster Recovery Workflow

Incident Detected

↓

Incident Assessment

↓

Activate DR Plan

↓

Restore Infrastructure

↓

Restore Database

↓

Restore Storage

↓

Application Startup

↓

Verification

↓

Business Resumes

14.12 Failure Scenarios

Supported Recovery

- Application Failure
 - Database Failure
 - Redis Failure
 - Storage Failure
 - Network Failure
 - Server Failure
 - Human Error
 - Data Corruption
-

14.13 Application Recovery

Application Failure

↓

Docker Restart

↓

Health Check

↓

Traffic Restored

Future

- Kubernetes Self-Healing
 - Automatic Pod Recovery
-

14.14 Database Recovery

Recovery Steps

1. Stop Application
 2. Restore Database Backup
 3. Validate Data
 4. Restart Services
 5. Verify Application
 6. Resume Traffic
-

14.15 Storage Recovery

Restore

- Resume Files
- Images
- Certificates
- Reports

Validation

- File Integrity
 - Metadata Consistency
 - Access Verification
-

14.16 Business Continuity Plan (BCP)

Critical Business Functions

- User Login
- Job Search
- Job Application
- Assessments
- Recruiter Operations

- Admin Operations
- Reports

Priority

Critical

↓

High

↓

Medium

↓

Low

14.17 Incident Classification

Severity	Example	Response
Critical	Complete platform outage	Immediate
High	Database unavailable	Within 30 minutes
Medium	AI service unavailable	Within 2 hours
Low	Minor UI issue	Next release

14.18 Recovery Team

Roles

- Incident Manager
- DevOps Engineer
- Database Administrator
- Backend Lead
- Security Engineer
- QA Lead
- Product Owner

Responsibilities

- Coordinate recovery

- Validate restoration
 - Communicate status
 - Approve production readiness
-

14.19 Communication Plan

Notify

- Operations Team
- Technical Team
- Management
- Customer Support

Future

- Public Status Page
 - Customer Notifications
-

14.20 Disaster Recovery Testing

Testing Schedule

Test	Frequency
Backup Verification	Weekly
Database Restore	Monthly
Application Recovery	Quarterly
Full DR Drill	Every 6 Months

Testing ensures recovery procedures remain effective.

14.21 Backup Retention Policy

Backup Type	Retention
Hourly/Incremental	7 Days
Daily	30 Days
Weekly	3 Months
Monthly	1 Year

Backup Type	Retention
Annual Archive	7 Years

14.22 Multi-Site Disaster Recovery (Future)

Primary Data Center

↓

Replication

↓

Secondary Data Center

↓

Recovery

↓

Business Continuity

Future Enhancements

- Active-Passive Architecture
 - Cross-Region Replication
 - Automatic Failover
-

14.23 Backup Security

All backups shall be:

- Encrypted
 - Access Controlled
 - Integrity Verified
 - Versioned
 - Protected from unauthorized modification or deletion
-

14.24 Recovery Validation

After restoration verify:

- Database Integrity
 - File Availability
 - User Authentication
 - API Functionality
 - Reports
 - AI Integration
 - Notifications
 - Performance
-

14.25 Business Continuity Workflow

Disaster

↓

Detection

↓

Containment

↓

Recovery

↓

Validation

↓

Production

↓

Post-Incident Review

14.26 Operational Runbook

Standard recovery checklist:

1. Detect and classify the incident.
 2. Inform the incident response team.
 3. Activate the disaster recovery plan.
 4. Restore infrastructure components.
 5. Recover databases and object storage.
 6. Validate application functionality.
 7. Resume production traffic.
 8. Monitor platform stability.
 9. Document lessons learned.
-

14.27 Future Roadmap

Phase 2

- Cross-region backups
- Automated failover
- Kubernetes self-healing
- Continuous replication

Phase 3

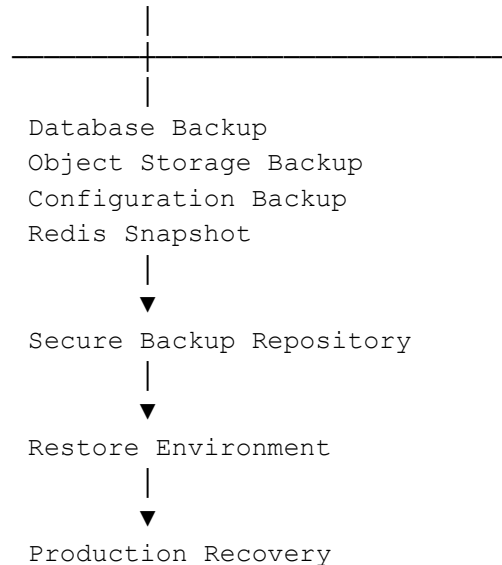
- Active-Active deployment
 - Multi-cloud disaster recovery
 - Automated DR orchestration
 - AI-assisted incident response
-

14.28 Best Practices

- Automate backup creation and verification.
 - Test disaster recovery procedures regularly.
 - Encrypt backup data in transit and at rest.
 - Store backups in separate physical or cloud locations.
 - Maintain documented recovery procedures.
 - Continuously monitor backup health.
 - Review RPO and RTO targets periodically.
 - Conduct post-incident reviews to improve resilience.
-

14.29 Disaster Recovery Architecture

Production Environment



14.30 Chapter Summary

This chapter defines the **Disaster Recovery, Backup & Business Continuity Architecture** for the Smart Education Platform. It establishes a comprehensive recovery strategy covering automated backups, disaster recovery workflows, business continuity planning, recovery objectives (RPO/RTO), backup security, operational runbooks, and future multi-region resilience. The architecture ensures that critical platform services can be restored efficiently while protecting business data and minimizing operational disruption.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 15 – Non-Functional Architecture & Quality Attributes

Document Version: 1.0

Architecture Style: Enterprise Quality Attribute Driven Architecture

15.1 Introduction

Non-Functional Requirements (NFRs) define how the Smart Education Platform should perform, scale, remain secure, and operate under different business and technical conditions.

Unlike functional requirements, which describe **what** the system does, non-functional requirements define **how well** it performs.

This chapter establishes the enterprise quality standards that guide the platform's architecture, implementation, deployment, and operations.

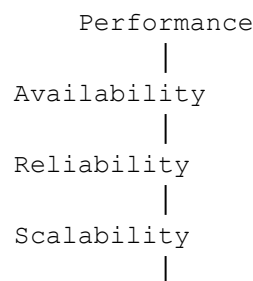
15.2 Objectives

The Non-Functional Architecture shall ensure:

- High Performance
 - High Availability
 - Scalability
 - Reliability
 - Security
 - Maintainability
 - Extensibility
 - Portability
 - Accessibility
 - Usability
 - Interoperability
 - Observability
 - Compliance
-

15.3 Quality Attribute Model

Quality Attributes



Security
|
Maintainability
|
Extensibility
|
Observability
|
Compliance

15.4 Performance

The platform shall deliver responsive user experiences.

Performance Targets

Requirement	Target
Login	< 2 sec
Dashboard	< 2 sec
API Response	< 300 ms
Database Query	< 100 ms
Redis Query	< 10 ms
Resume Upload	< 5 sec
AI Response	< 10 sec

Optimization Techniques

- Redis Caching
 - Query Optimization
 - Pagination
 - Lazy Loading
 - Connection Pooling
 - Compression
 - Asynchronous Processing
-

15.5 Availability

Target Availability

99.9%

Availability is achieved through

- Redundant Application Instances
 - Health Checks
 - Monitoring
 - Automated Recovery
 - Backup Strategy
 - Disaster Recovery
-

15.6 Reliability

The platform must consistently perform correctly under expected workloads.

Reliability Measures

- ACID Transactions
 - Error Recovery
 - Retry Policies
 - Circuit Breakers
 - Graceful Error Handling
 - Backup Verification
-

15.7 Scalability

The architecture supports both vertical and horizontal scaling.

Phase 1

- Docker
- Modular Monolith

Phase 2

- Kubernetes
- Read Replicas
- Auto Scaling

Phase 3

- Microservices
 - Multi-Region Deployment
-

15.8 Security

Security Principles

- Zero Trust
 - RBAC
 - JWT Authentication
 - HTTPS
 - TLS 1.3
 - AES Encryption
 - Audit Logging
 - Secure File Storage
 - OWASP Compliance
-

15.9 Maintainability

Maintainability is achieved by

- Modular Architecture
 - Clean Code
 - SOLID Principles
 - Design Patterns
 - Layered Architecture
 - Standardized APIs
 - Automated Testing
-

15.10 Extensibility

The platform is designed for future expansion.

Future Modules

- Learning Management
- Live Classes

- Mentor Portal
- Parent Portal
- Multi-Tenant Support
- Payment Gateway

No major redesign should be required to introduce these features.

15.11 Portability

Supported Platforms

Web

- Chrome
- Firefox
- Edge
- Safari

Mobile

- Android

Future

- iOS

Deployment Platforms

- AWS
 - Azure
 - GCP
 - OCI
 - DigitalOcean
-

15.12 Accessibility

The platform targets **WCAG 2.1 AA** compliance.

Supports

- Keyboard Navigation
 - Screen Readers
 - High Contrast
 - Responsive Text
 - Focus Indicators
 - Minimum Touch Area (44 × 44 px)
-

15.13 Usability

The user experience emphasizes:

- Simple Navigation
 - Consistent UI
 - Fast Workflows
 - Clear Feedback
 - Minimal User Actions
 - Responsive Design
 - Error Prevention
-

15.14 Interoperability

The platform supports integration using:

- REST APIs
 - JSON
 - HTTPS
 - JWT
 - Webhooks (Future)
 - OAuth 2.0 (Future)
 - OpenID Connect (Future)
-

15.15 Observability

Observability includes:

- Metrics
- Logs

- Alerts
- Dashboards
- Health Checks
- Audit Trails

Tools

- Prometheus
- Grafana
- Spring Boot Actuator
- Logback

Future

- OpenTelemetry
 - ELK Stack
-

15.16 Testability

The architecture supports

- Unit Testing
- Integration Testing
- API Testing
- UI Testing
- Performance Testing
- Security Testing

Target Code Coverage

- Minimum 80%
-

15.17 Recoverability

Recovery Mechanisms

- Database Backup
- Docker Recovery
- Configuration Restore
- Object Storage Restore

- Disaster Recovery Plan

Recovery Targets

Metric Target

RPO 15 Minutes

RTO 1 Hour

15.18 Compliance

The platform aligns with:

- OWASP Top 10
- Secure Coding Practices
- Password Security Standards
- Data Privacy Best Practices

Future Readiness

- ISO 27001
 - SOC 2
-

15.19 Capacity Planning

Expected Phase 1 Capacity

Resource	Target
Learners	100,000+
Recruiters	5,000+
Super Admins	50+
Concurrent Users	5,000+
Daily API Requests	2 Million+
Daily AI Requests	100,000+

15.20 Fault Tolerance

Supported Mechanisms

- Retry Logic
- Circuit Breaker
- Graceful Degradation
- Health Checks
- Automatic Restart

Future

- Kubernetes Self-Healing
-

15.21 Data Integrity

The platform ensures data integrity through:

- ACID Transactions
 - Foreign Key Constraints
 - Optimistic Locking
 - Validation Rules
 - Audit Trails
 - Transaction Management
-

15.22 Configuration Management

Configuration Categories

- Database
- Redis
- MinIO
- AI Provider
- Email
- SMS
- Security
- Logging

Environment-specific configuration is isolated for Development, QA, UAT, and Production.

15.23 Quality Attribute Scenarios

Attribute	Scenario	Expected Outcome
Performance	5,000 concurrent users	Response time within SLA
Availability	One application instance fails	Traffic routed to healthy instance
Security	Invalid JWT	Request rejected with 401 Unauthorized
Reliability	Database reconnect required	Automatic recovery without data corruption
Scalability	Traffic doubles	New application instances added (future)
Recoverability	Database restored	Service resumes within RTO

15.24 Architecture Trade-offs

Decision	Benefit	Trade-off
Modular Monolith	Simpler development	Less independent scaling than microservices
PostgreSQL	Strong consistency	Vertical scaling limits before sharding
Redis Cache	Faster response times	Cache invalidation complexity
Docker	Consistent deployment	Container management overhead
JWT Authentication	Stateless APIs	Token revocation requires additional handling

15.25 Enterprise Readiness Assessment

The platform is designed to be enterprise-ready with:

- Modular Architecture
 - Secure Authentication
 - API-First Design
 - Cloud Readiness
 - Automated Deployment
 - Monitoring & Observability
 - Disaster Recovery
 - Backup Strategy
 - Role-Based Access Control
 - AI Integration
-

15.26 Architecture Validation Checklist

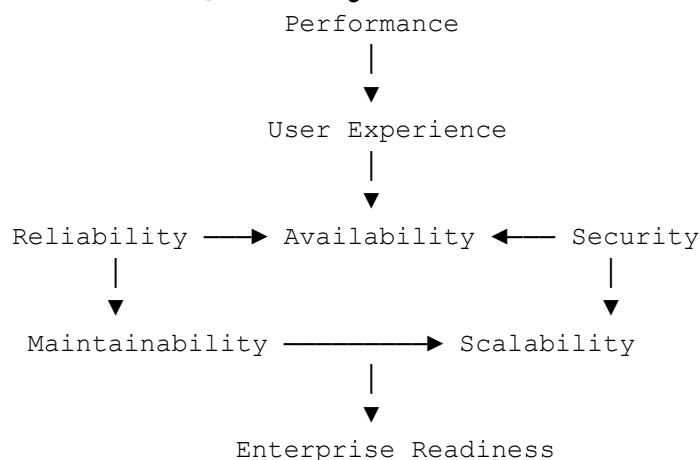
The architecture should be validated for:

- Functional Completeness
 - Performance Targets
 - Security Controls
 - Scalability Readiness
 - Disaster Recovery
 - Accessibility Compliance
 - Monitoring Coverage
 - Documentation Completeness
-

15.27 Best Practices

- Design for scalability from the beginning.
 - Optimize performance using caching and efficient database access.
 - Apply security at every architectural layer.
 - Monitor system health continuously.
 - Automate testing and deployments.
 - Keep modules loosely coupled.
 - Review quality attributes regularly as the platform evolves.
 - Validate non-functional requirements before every production release.
-

15.28 Quality Attribute Relationship



15.29 Chapter Summary

This chapter defines the **Non-Functional Architecture & Quality Attributes** of the Smart Education Platform. It establishes measurable quality goals for performance, scalability, availability, reliability, security, maintainability, extensibility, accessibility, interoperability, observability, and recoverability. By defining concrete targets, architectural trade-offs, and validation criteria, the platform is positioned to deliver a secure, scalable, and enterprise-grade user experience while supporting future growth and continuous evolution.

High-Level Design (HLD)

Smart Education Platform (Phase 1)

Chapter 16 – Technology Decision Matrix, Architecture Evolution & Future Roadmap

Document Version: 1.0

Architecture Style: Enterprise Modular Monolith → Cloud Native Microservices

16.1 Introduction

This chapter documents the **technology decisions**, **architectural evolution**, and **long-term roadmap** of the Smart Education Platform.

Every technology has been selected after considering:

- Scalability
- Performance
- Security
- Maintainability
- Community Support
- Enterprise Adoption
- Cost
- Future Growth

The goal is to build an enterprise-grade platform that can grow from a startup deployment to a large-scale education ecosystem without major architectural redesign.

16.2 Architecture Vision

The Smart Education Platform will evolve through three major phases.

Phase 1
Enterprise Modular Monolith

↓

Phase 2
Cloud Native Architecture

↓

Phase 3
Enterprise Microservices Platform

16.3 Technology Decision Principles

Technology selection is based on:

- Open Source First
 - Enterprise Stability
 - Long-Term Support (LTS)
 - Large Developer Community
 - Security
 - High Performance
 - Easy Maintenance
 - Cloud Compatibility
 - Vendor Neutrality
-

16.4 Technology Decision Matrix

Layer	Selected Technology	Alternative	Reason
Backend	Spring Boot	Quarkus, Micronaut	Mature ecosystem, enterprise adoption
Programming Language	Java 21 LTS	Kotlin, Go	Stability, LTS, large talent pool
Frontend	React	Angular, Vue	Rich ecosystem and component reusability

Layer	Selected Technology	Alternative	Reason
Mobile	Flutter	React Native	Single codebase for Android/iOS
Database	PostgreSQL	MySQL	Advanced SQL, reliability, extensibility
Cache	Redis	Memcached	Rich data structures and performance
Object Storage	MinIO	AWS S3	S3-compatible, self-hosted, cost-effective
Security	Spring Security + JWT	Keycloak	Lightweight and customizable for Phase 1
Reverse Proxy	Nginx	Apache	High performance and low resource usage
Build Tool	Maven	Gradle	Enterprise standard for Spring ecosystem
Containerization	Docker	Podman	Mature tooling and broad adoption
Monitoring	Prometheus + Grafana	Datadog	Open source and extensible
CI/CD	GitHub Actions	Jenkins	Native GitHub integration and simplicity

16.5 Backend Technology Stack

Core Technologies

- Java 21 LTS
- Spring Boot
- Spring Security
- Spring Data JPA
- Hibernate
- Spring Validation
- Spring Scheduler

Benefits

- Enterprise-grade architecture
- Modular development
- High performance
- Strong ecosystem

- Long-term maintainability
-

16.6 Frontend Technology Stack

Web

- React
- TypeScript
- Redux Toolkit
- React Router
- Axios

Mobile

- Flutter
- Dart

Benefits

- Component-based architecture
 - Responsive UI
 - Cross-platform development
 - Easy maintenance
-

16.7 Data Layer Technology

Primary Database

- PostgreSQL

Cache

- Redis

Object Storage

- MinIO

Advantages

- High reliability

- ACID compliance
 - Fast caching
 - Secure file storage
-

16.8 Infrastructure Technology

Infrastructure Stack

- Ubuntu Server
- Docker
- Nginx
- GitHub Actions
- Prometheus
- Grafana

Future

- Kubernetes
 - Helm
 - Terraform
-

16.9 Security Technology

Selected Components

- Spring Security
- JWT Authentication
- BCrypt Password Hashing
- TLS 1.3
- HTTPS
- RBAC

Future

- OAuth2
 - OpenID Connect
 - Multi-Factor Authentication (MFA)
 - Web Application Firewall (WAF)
-

16.10 AI Technology Roadmap

Phase 1

- AI Coach
- Resume Review
- Interview Guidance
- Coding Assistance
- SQL Assistance

Phase 2

- Personalized Career Recommendations
- Adaptive Practice Suggestions
- AI Performance Analytics

Phase 3

- Intelligent Placement Assistant
 - AI Learning Companion
 - Predictive Career Insights
-

16.11 Architecture Evolution

Phase 1 – Modular Monolith

Spring Boot

|— Authentication
|— Assessment
|— Gamification
|— AI Coach
|— Placement
|— Reports
|— Notifications

Characteristics

- Faster development
 - Simplified deployment
 - Lower operational complexity
-

Phase 2 – Cloud Native

API Gateway

↓

Container Platform

↓

Independent Modules

↓

Shared Services

Features

- Docker
 - Kubernetes
 - Horizontal Scaling
 - API Gateway
-

Phase 3 – Microservices

API Gateway

```
|— Identity Service
|— User Service
|— Assessment Service
|— AI Service
|— Placement Service
|— Recruiter Service
|— Notification Service
|— Analytics Service
```

Benefits

- Independent deployment
 - Independent scaling
 - Technology flexibility
 - Better fault isolation
-

16.12 Architecture Decision Records (ADR)

ADR	Decision	Rationale
ADR-001	Modular Monolith	Faster delivery for Phase 1
ADR-002	Spring Boot	Enterprise ecosystem
ADR-003	PostgreSQL	Strong consistency and scalability
ADR-004	Redis	High-performance caching
ADR-005	MinIO	Self-hosted object storage
ADR-006	Docker	Consistent deployments
ADR-007	React + Flutter	Unified modern user experience

16.13 Phase-wise Product Roadmap

Phase 1 (Current)

- Independent Learner Portal
 - Recruiter Portal
 - Super Admin Portal
 - Gamification
 - Brain Gym
 - Aptitude Arena
 - Coding Arena
 - SQL Arena
 - Debug Hero
 - AI Coach
 - Resume Builder
 - Placement Portal
 - Reports & Analytics
-

Phase 2

- Learning Module
- Live Coding Contests
- Mentor Portal
- Company Assessments
- API Gateway
- Kubernetes
- Event Streaming

Phase 3

- LMS Integration
 - Parent Portal
 - College Portal
 - Multi-Tenant Architecture
 - AI Recommendation Engine
 - Enterprise Analytics
 - Mobile iOS Application
 - Global Deployment
-

16.14 Cloud Evolution Roadmap

Local Server

↓

Cloud VM

↓

Docker

↓

Kubernetes

↓

Multi-Region Cloud

↓

Global Platform

16.15 Scalability Roadmap

Phase **Expected Capacity**

Phase 1 100,000+ Learners

Phase 2 500,000+ Learners

Phase Expected Capacity

Phase 3 5 Million+ Learners

16.16 Enterprise Expansion Roadmap

Future Platform Modules

- Learning Management System (LMS)
 - Mentor Portal
 - Parent Portal
 - College Administration Portal
 - Training Institute Portal
 - Enterprise HR Portal
 - Corporate Assessment Platform
 - Certification Management
 - Internship Management
 - Alumni Network
-

16.17 Technology Upgrade Strategy

Regularly review and upgrade:

- Java LTS Versions
- Spring Boot Releases
- React & Flutter Versions
- PostgreSQL
- Redis
- Docker
- Security Libraries
- Monitoring Stack

Upgrade Process

1. Evaluate compatibility
2. Test in Development
3. Validate in QA/UAT
4. Deploy to Production
5. Monitor post-release

16.18 Future Enterprise Integrations

Planned integrations include:

- LinkedIn
 - GitHub
 - Google Workspace
 - Microsoft 365
 - Google Calendar
 - Zoom
 - Microsoft Teams
 - WhatsApp Business API
 - Payment Gateways
 - HRMS & ERP Systems
-

16.19 Risks & Mitigation

Risk	Mitigation
Rapid user growth	Horizontal scaling and caching
Third-party API failure	Retry, timeout, circuit breaker
Database bottleneck	Read replicas and optimization
Security threats	Continuous monitoring and regular security updates
Infrastructure failure	Backup, DR, and high availability architecture

16.20 Enterprise Readiness Checklist

The platform is designed with:

- Modular Architecture
- API-First Design
- Cloud Readiness
- Containerization
- Secure Authentication
- CI/CD Pipeline
- Monitoring & Observability

- Disaster Recovery
 - AI Integration
 - Scalability Roadmap
-

16.21 Long-Term Vision

The Smart Education Platform aims to become a unified ecosystem for:

- Students
 - Independent Learners
 - Recruiters
 - Colleges
 - Universities
 - Training Institutes
 - Corporate HR Teams
 - Government Skill Development Programs
-

16.22 Architecture Maturity Model

Level 1

Foundation Platform

↓

Level 2

Enterprise Platform

↓

Level 3

Cloud Native Platform

↓

Level 4

AI-Driven Intelligent Platform

↓

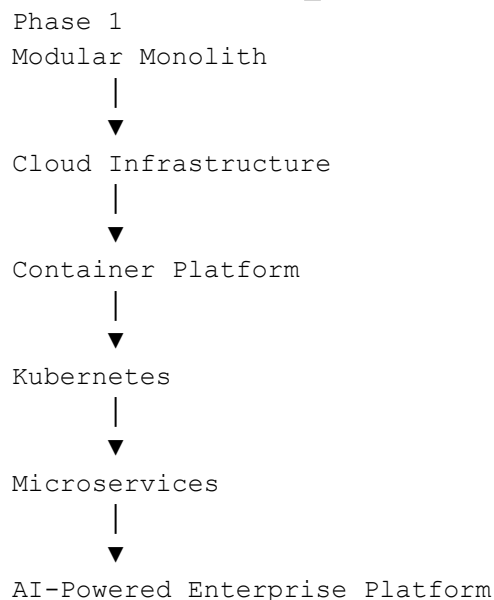
Level 5

Global Education Ecosystem

16.23 Best Practices

- Prefer proven enterprise technologies.
 - Upgrade dependencies regularly.
 - Keep architecture modular and loosely coupled.
 - Make technology decisions based on business needs.
 - Validate architectural changes with ADRs.
 - Plan scalability before it becomes necessary.
 - Invest in automation, monitoring, and security from the beginning.
-

16.24 Complete Architecture Evolution



16.25 Final Architecture Recommendations

- Continue with a **Modular Monolith** during Phase 1 to accelerate development.
- Standardize all APIs using REST and OpenAPI specifications.
- Build every module with clear separation of concerns to simplify future extraction into microservices.
- Adopt containerized deployments from the start.
- Monitor platform health continuously using Prometheus and Grafana.
- Prioritize security, observability, and automation throughout the development lifecycle.
- Introduce Kubernetes and an API Gateway only when operational scale justifies the added complexity.

16.26 HLD Conclusion

The **High-Level Design (HLD)** provides a complete enterprise architecture blueprint for the **Smart Education Platform (Phase 1)**. It defines the platform's vision, architecture, technology stack, deployment strategy, security model, scalability roadmap, integration approach, operational readiness, and long-term evolution.

Together with the **PRD, SRS, LLD, UI/UX Design System, Database Design, REST API Specification, Implementation Plan, Testing Strategy, and Operations Runbook**, this HLD forms a comprehensive technical foundation for designing, developing, deploying, and operating the platform.